

분산처리

GPU Architecture & CUDA Programming

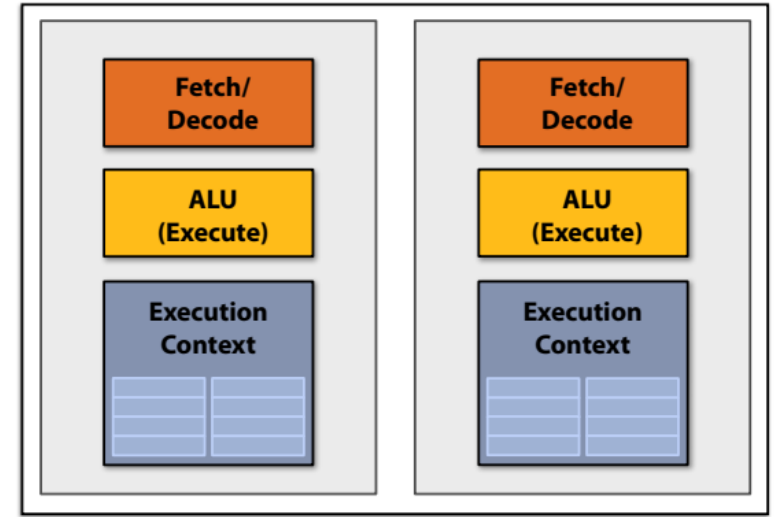


Computer & Ai

Department of Computer Engineering

Review: how to run code on a CPU

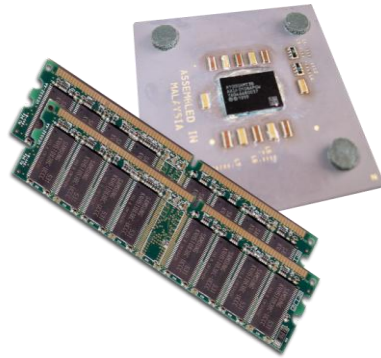
- 사용자가 멀티코어 CPU에서 프로그램을 실행하려고 한다고 가정해 보자
- ✓ OS가 프로그램 텍스트를 메모리에 로드.
- ✓ OS가 CPU 실행 컨텍스트 선택
- ✓ OS가 프로세서를 중단하고 실행 컨텍스트를 준비
(레지스터, 프로그램 카운터 등의 내용을 설정하여 실행 컨텍스트 준비).
- ✓ 실행!
- ✓ 프로세서는 실행 컨텍스트에서 유지된 환경 내에서 명령어 실행을 시작.



Multi-core CPU

Heterogeneous Computing

- Terminology:
 - *Host* The CPU and its memory (host memory)
 - *Device* The GPU and its memory (device memory)



Host



Device

Heterogeneous Computing

```
#include <iostream>
#include <algorithm>

using namespace std;

#define N 1024
#define RADIUS 3
#define BLOCK_SIZE 16

__global__ void stencil_1d(int *in, int *out) {
    __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];
    int gindex = threadIdx.x + blockIdx.x * blockDim.x;
    int lindex = threadIdx.x + RADIUS;

    // Read input elements into shared memory
    temp[lindex] = in[gindex];
    if (threadIdx.x < RADIUS) {
        temp[lindex - RADIUS] = in[gindex - RADIUS];
        temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
    }

    // Synchronize (ensure all the data is available)
    __syncthreads();

    // Apply the stencil
    int result = 0;
    for (int offset = -RADIUS; offset <= RADIUS; offset++)
        result += temp[lindex + offset];

    // Store the result
    out[gindex] = result;
}

void fill_ints(int *x, int n) {
    fill_n(x, n, 1);
}

int main(void) {
    int *in, *out; // host copies of a, b, c
    int *d_in, *d_out; // device copies of a, b, c
    int size = (N + 2*RADIUS) * sizeof(int);

    // Alloc space for host copies and setup values
    in = (int *)malloc(size); fill_ints(in, N + 2*RADIUS);
    out = (int *)malloc(size); fill_ints(out, N + 2*RADIUS);

    // Alloc space for device copies
    cudaMalloc((void **)&d_in, size);
    cudaMalloc((void **)&d_out, size);

    // Copy to device
    cudaMemcpy(d_in, in, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_out, out, size, cudaMemcpyHostToDevice);

    // Launch stencil_1d() kernel on GPU
    stencil_1d<<<N/BLOCK_SIZE,BLOCK_SIZE>>>(d_in + RADIUS, d_out + RADIUS);

    // Copy result back to host
    cudaMemcpy(out, d_out, size, cudaMemcpyDeviceToHost);

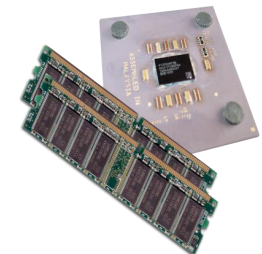
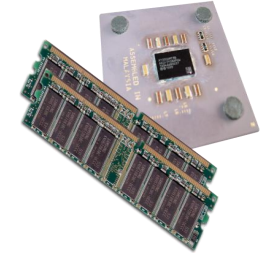
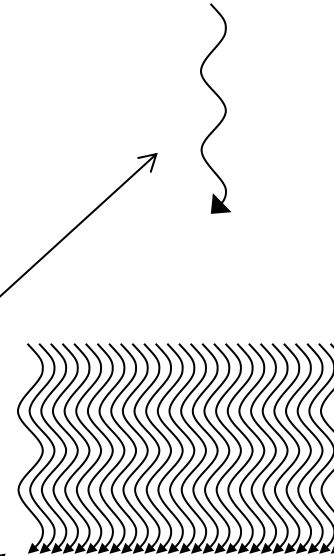
    // Cleanup
    free(in); free(out);
    cudaFree(d_in); cudaFree(d_out);
    return 0;
}
```

parallel fn

serial code

parallel code

serial code



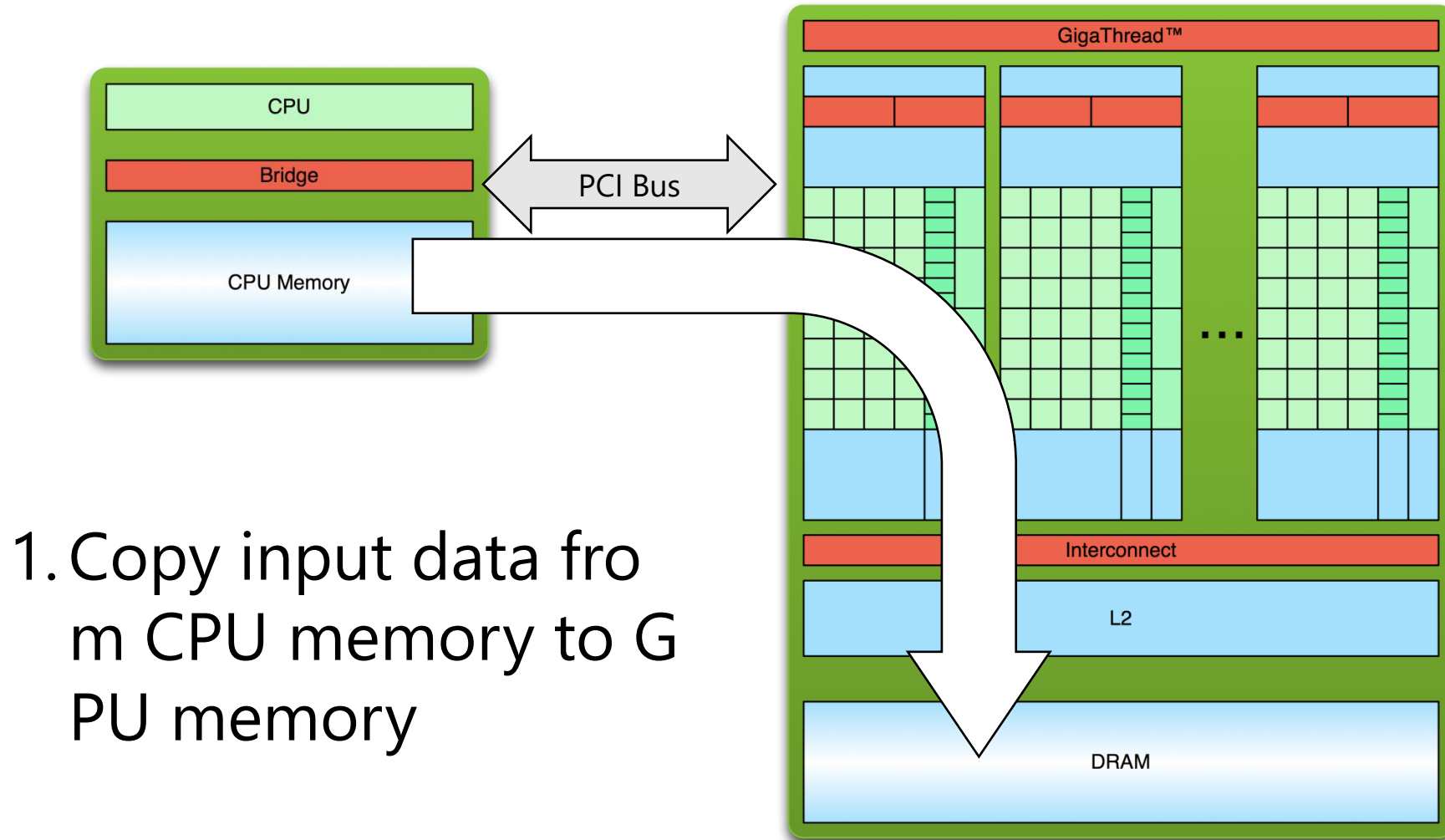
Heterogeneous Computing

<https://github.com/PacktPublishing/Learn-CUDA-Programming>

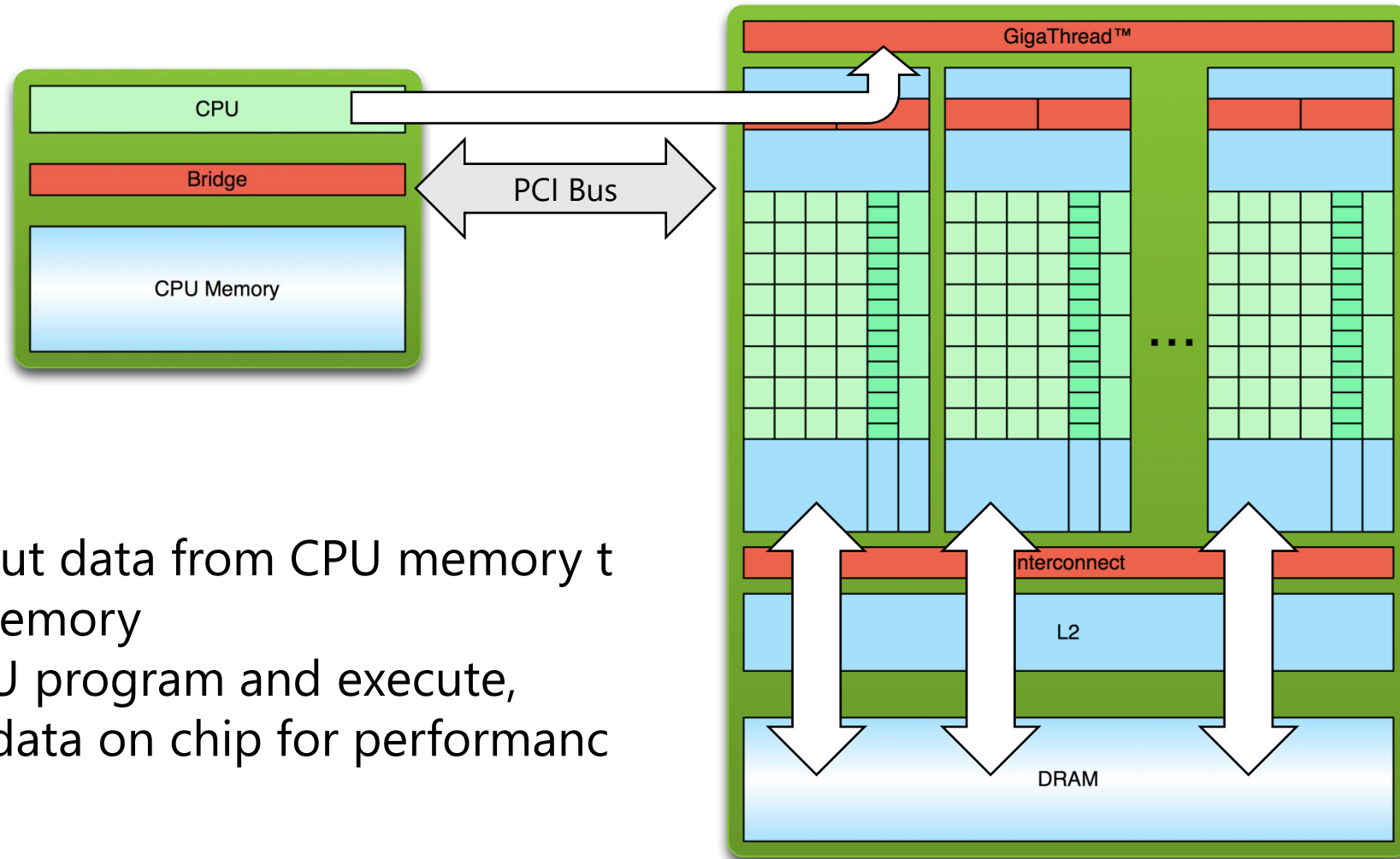
<https://github.com/CUDA-Tutorial/CodeSamples>

<https://cuda-tutorial.github.io/>

Simple Processing Flow

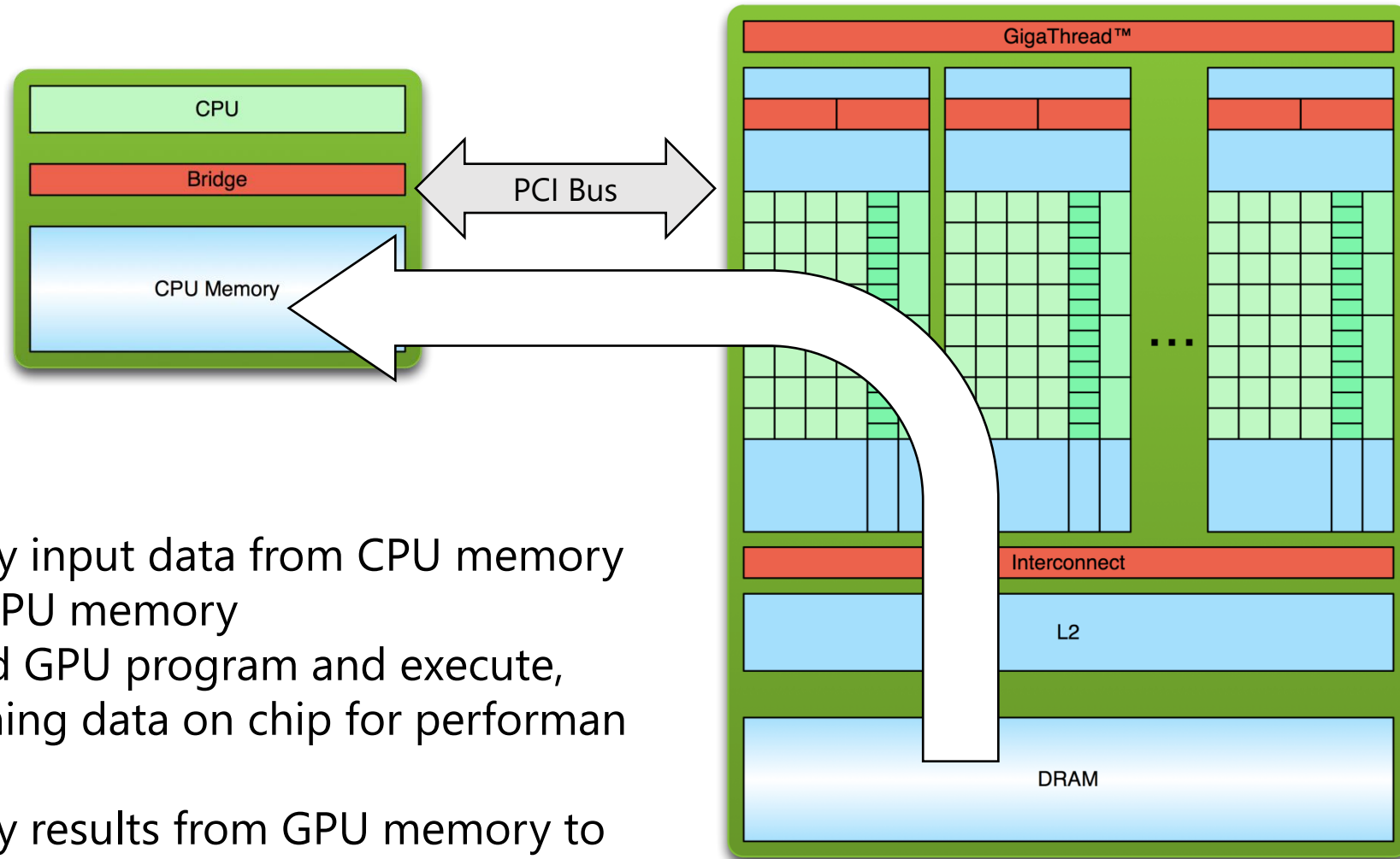


Simple Processing Flow



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance

Simple Processing Flow



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance
3. Copy results from GPU memory to CPU memory

Hello World!

```
int main(void) {  
    printf("Hello World!\n");  
    return 0;  
}
```

- 호스트에서 실행되는 표준 C
- 장치 코드가 없는 프로그램을 컴파일하는 데 NVIDIA 컴파일러(nvcc)를 사용할 수 있다.

Output:

```
$ nvcc hello_world.cu  
$ a.out  
Hello World!  
$
```

Hello World! with Device Code

```
__global__ void mykernel(void) {  
}  
  
int main(void) {  
    mykernel<<<1,1>>>();  
    printf("Hello World!\n");  
    return 0;  
}
```

- 두 가지 새로운 구문 요소...

Hello World! with Device Code

```
__global__ void mykernel(void) {  
}
```

- CUDA C/C++ keyword `__global__` 함수를 나타냄:
 - 장치(GPU)에서 실행됨
 - 호스트 코드에서 호출됨
- 소스 코드를 호스트 및 디바이스 구성 요소로 분리하는 NVCC
 - NVIDIA 컴파일러가 처리하는 디바이스 함수
(예: `mykernel()`)
 - 표준 호스트 컴파일러에 의해 처리되는 호스트 함수
(예: `main()`)
 - `gcc`, `cl.exe`

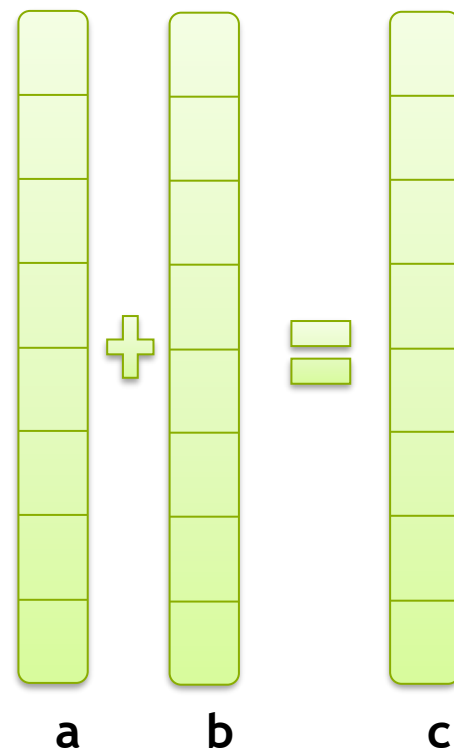
Hello World! with Device Code

```
mykernel<<<1,1>>>();
```

- 삼중 꺾쇠 괄호는 호스트 코드에서 디바이스 코드로의 호출을 표시.
 - "커널 실행"이라고도 함.
 - 잠시 후 매개변수(1,1)로 돌아감.
- 이것이 GPU에서 함수를 실행하는 데 필요한 모든 것 !

Parallel Programming in CUDA C/C++

- 하지만 잠깐만요... GPU 컴퓨팅은 대규모 병렬 처리에 관한 것입니다!
- 좀 더 흥미로운 예제가 필요합니다...
- 두 개의 정수를 더하는 것부터 시작해서 벡터 덧셈까지 확장해 보겠습니다.



Addition on the Device

- A simple kernel to add two integers

```
__global__ void add(int *a, int *b, int *c) {  
    *c = *a + *b;  
}
```

- As before `__global__` is a CUDA C/C++ keyword meaning
 - `add()` 가 디바이스에서 실행.
 - `add()` 는 호스트에서 호출.

Addition on the Device

- Note that we use pointers for the variables

```
__global__ void add(int *a, int *b, int *c) {  
    *c = *a + *b;  
}
```

- `add()` runs on the device, so `a`, `b` and `c` must point to device memory
- We need to allocate memory on the GPU

Memory Management

- 호스트와 디바이스 메모리는 별도의 엔티티입니다.

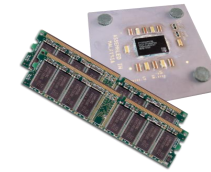
- *Device* pointers 는 GPU 메모리를 가리킴

- 호스트 코드와 전달될 수 있음
- 호스트 코드에서 역참조할 수 없음



- *Host* 포인터가 CPU 메모리를 가리킴

- 디바이스 코드에서 디바이스 코드에 전달될 수 있음
- 디바이스 코드에서 역참조할 수 없음



- 디바이스 메모리 처리를 위한 간단한 CUDA API

- cudaMalloc(), cudaFree(), cudaMemcpy()
- C와 동등한 malloc(), free(), memcpy()와 유사.

Addition on the Device: add()

- add() kernel 돌아가기

```
__global__ void add(int *a, int *b, int *c) {  
    *c = *a + *b;  
}
```

- main() 함수를 살펴봅시다...

Addition on the Device: main()

```
int main(void) {  
    int a, b, c;                // host copies of a, b, c  
    int *d_a, *d_b, *d_c;      // device copies of a, b, c  
    int size = sizeof(int);  
  
    // Allocate space for device copies of a, b, c  
    cudaMalloc((void **)&d_a, size);  
    cudaMalloc((void **)&d_b, size);  
    cudaMalloc((void **)&d_c, size);  
  
    // Setup input values  
    a = 2;  
    b = 7;
```

Addition on the Device: main()

```
// Copy inputs to device
cudaMemcpy(d_a, &a, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, &b, size, cudaMemcpyHostToDevice);

// Launch add() kernel on GPU
add<<<1,1>>>(d_a, d_b, d_c);

// Copy result back to host
cudaMemcpy(&c, d_c, size, cudaMemcpyDeviceToHost);

// Cleanup
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
return 0;
}
```

Parallel로의 이동

- GPU 컴퓨팅은 대규모 병렬 처리에 관한 것입니다. 그렇다면 디바이스에서 코드를 병렬로 실행하려면 어떻게 해야 할까요?

```
add<<< 1, 1 >>> ();
```

```
add<<< N, 1 >>> ();
```

- add()를 한 번 실행하는 대신 **N번 병렬로 실행.**

Vector Addition on the Device

- add()를 병렬로 실행하면 벡터 덧셈을 할 수 있음.
- 용어: add()의 각 병렬 호출을 블록이라고 함.
 - 블록의 집합을 그리드라고 함.
 - 각 호출은 `blockIdx.x`를 사용하여 해당 블록 인덱스를 참조할 수 있음.

```
__global__ void add(int *a, int *b, int *c) {  
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];  
}
```

- `blockIdx.x`를 사용하여 배열에 인덱싱하면 각 블록이 서로 다른 인덱스를 처리.

Vector Addition on the Device

```
__global__ void add(int *a, int *b, int *c) {  
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];  
}
```

- Device에서 각 블록은 병렬로 실행할 수 있음:

Block 0

```
c[0] = a[0] + b[0];
```

Block 1

```
c[1] = a[1] + b[1];
```

Block 2

```
c[2] = a[2] + b[2];
```

Block 3

```
c[3] = a[3] + b[3];
```

Vector Addition on the Device: add()

- 병렬화된 add() 커널로 돌아가기

```
__global__ void add(int *a, int *b, int *c) {  
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];  
}
```

- main()...을 살펴보자...

Vector Addition on the Device: main()

```
#define N 512
int main(void) {
    int *a  *b  *c                // host copies of a, b, c
    int *d_a, *d_b, *d_c; // device copies of a, b, c
    int size = N * sizeof(int);

    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Alloc space for host copies of a, b, c and setup input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);
```


Vector Addition on the Device: main()

```
// Copy inputs to device
cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

// Launch add() kernel on GPU with N blocks
add<<<N,1>>>(d_a, d_b, d_c);

// Copy result back to host
cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

// Cleanup
free(a); free(b); free(c);
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
return 0;
}
```

CUDA Threads

- Terminology(의미):
 - (블록)block을 병렬 스레드(parallel threads)로 분할 가능
 - 병렬 블록(parallel blocks) 대신 병렬 스레드(parallel threads)를 사용하도록 add()를 변경해 보자.

```
__global__ void add(int *a, int *b, int *c) {  
    c[threadIdx.x] = a[threadIdx.x] + b[threadIdx.x];  
}
```

- blockIdx.x 대신에 threadIdx.x 사용
- main() 한가지 변경이 필요..

Vector Addition Using Threads: main()

```
#define N 512
int main(void) {
    int *a, *b, *c;                // host copies of a, b, c
    int *d_a, *d_b, *d_c;         // device copies of a, b, c
    int size = N * sizeof(int);

    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Alloc space for host copies of a, b, c and setup input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);
```

Vector Addition Using Threads: main()

```
    // Copy inputs to device
    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    // Launch add() kernel on GPU with N threads
    add<<<1,N>>>(d_a, d_b, d_c);

    // Copy result back to host
    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

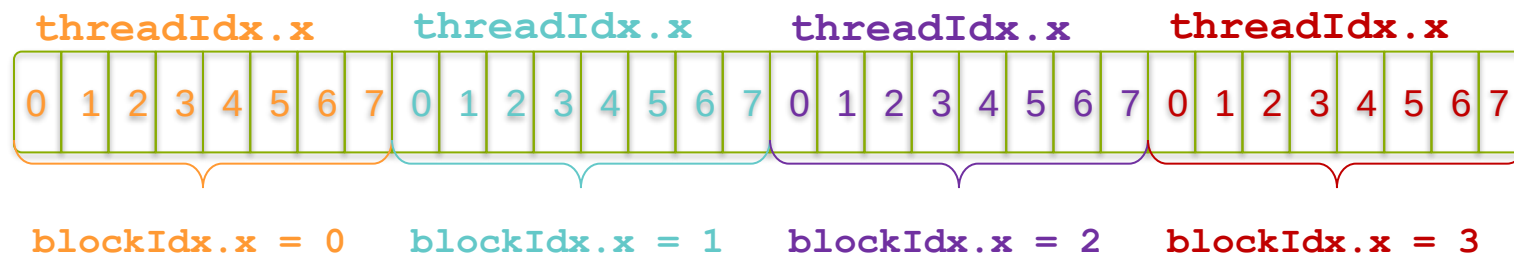
    // Cleanup
    free(a); free(b); free(c);
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}
```

Combining Blocks and Threads

- 병렬 벡터 덧셈을 사용해 보자:
 - 각각 하나의 스레드가 있는 많은 블록
 - 스레드가 많은 하나의 블록
- 블록과 스레드를 모두 사용하도록 벡터 덧셈을 조정해 보자.
- 왜 그럴까요? 그 이유에 대해 알아보자...
- 먼저 데이터 인덱싱(data indexing)에 대해 알아보자...

Indexing Arrays with Blocks and Threads

- `blockIdx.x` 과 `threadIdx.x` 사용이 간단하지 않음
 - 스레드당 하나의 요소(8개 스레드/블록)로 배열을 인덱싱한다고 가정.

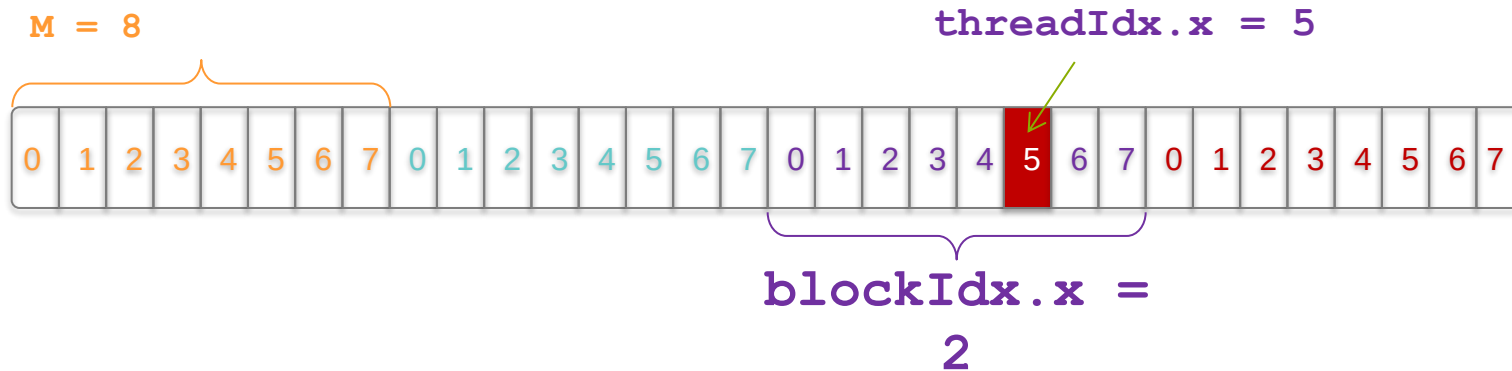
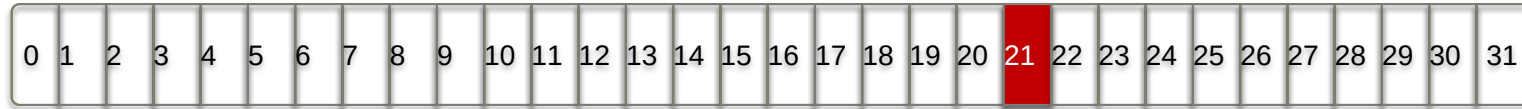


- M 스레드/블록의 경우 각 스레드에 대한 고유 인덱스는 다음과 같이 지정 됨:

```
int index = threadIdx.x + blockIdx.x * M;
```

Indexing Arrays: Example

- 빨간색 요소에서 어떤 스레드가 작동하나요?

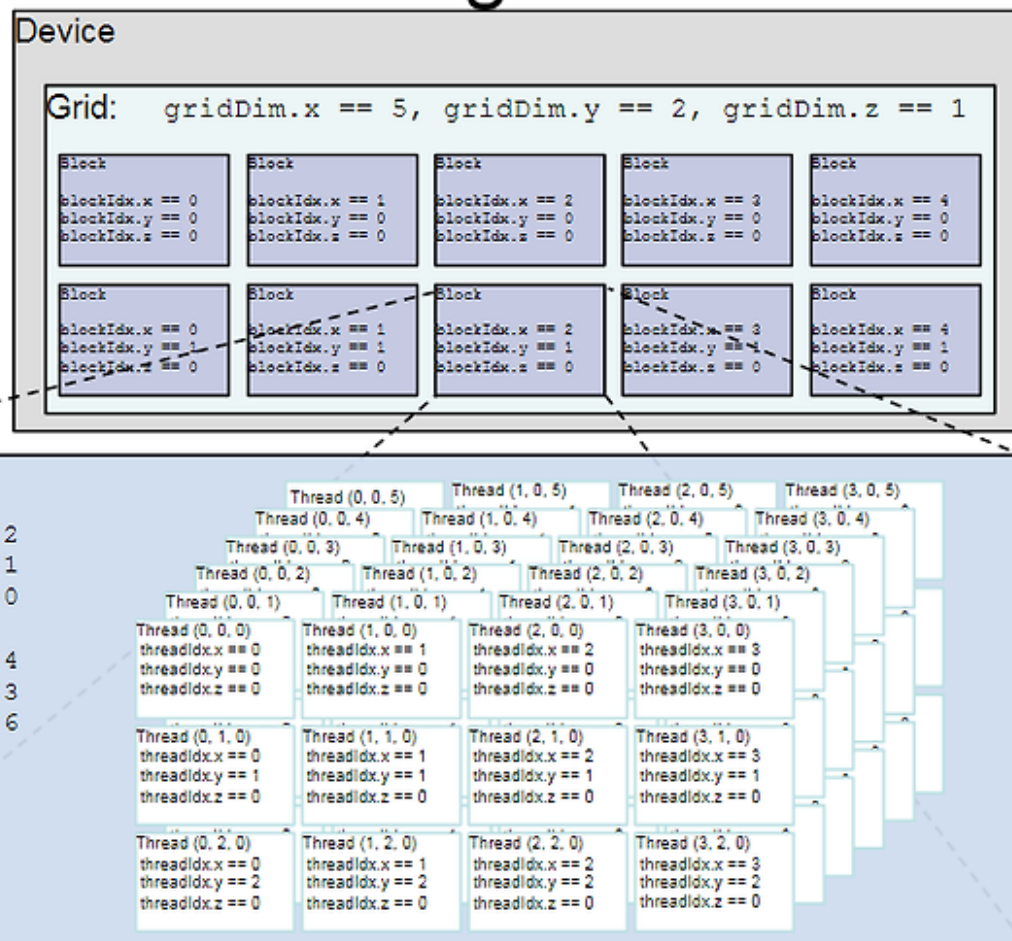


```
int index = threadIdx.x + blockIdx.x * M;  
          =          5      +          2      * 8;  
          = 21;
```

CUDA Thread Organization

```
dim3 dimGrid(5, 2, 1);  
dim3 dimBlock(4, 3, 1);
```

Kernel



Vector Addition with Blocks and Threads

- 블록당 스레드에 내장 변수 blockDim.x를 사용

```
int index = threadIdx.x + blockIdx.x * blockDim.x;
```

- 병렬 스레드와 병렬 블록을 사용하기 위한 add()의 결합 버전

```
__global__ void add(int *a, int *b, int *c) {  
    int index = threadIdx.x + blockIdx.x * blockDim.x;  
    c[index] = a[index] + b[index];  
}
```

- main()에서 어떤 변경이 필요하나요?

Addition with Blocks and Threads: main()

```
#define N (2048*2048)
#define THREADS_PER_BLOCK 512
int main(void) {
    int *a, *b, *c;                // host copies of a, b, c
    int *d_a, *d_b, *d_c;          // device copies of a, b, c
    int size = N * sizeof(int);

    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Alloc space for host copies of a, b, c and setup input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);
```

Addition with Blocks and Threads: main()

```
// Copy inputs to device
cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

// Launch add() kernel on GPU
add<<<N/THREADS_PER_BLOCK, THREADS_PER_BLOCK>>>(d_a, d_b, d_c);

// Copy result back to host
cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

// Cleanup
free(a); free(b); free(c);
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
return 0;
}
```

Handling Arbitrary Vector Sizes

- 일반적인 문제는 `blockDim.x`에 친숙하지 않는 배수이다.
- 배열을 넘어서는 접근(Access)은 피하자

```
__global__ void add(int *a, int *b, int *c, int n) {  
    int index = threadIdx.x + blockIdx.x * blockDim.x;  
    if (index < n)  
        c[index] = a[index] + b[index];  
}
```

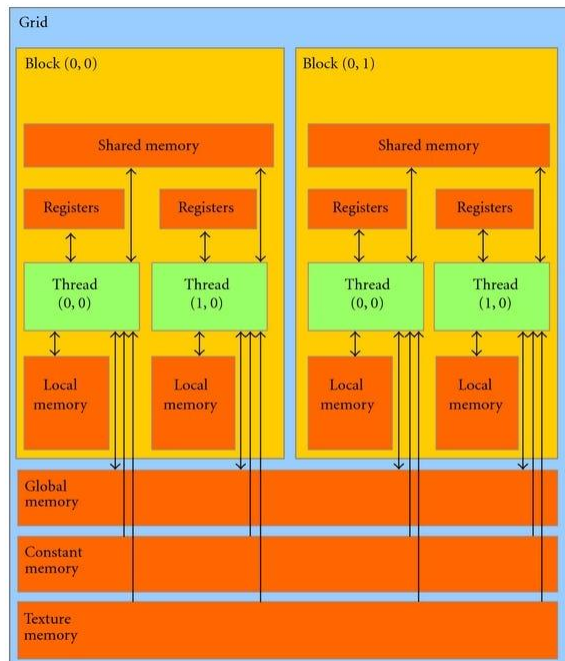
- Update the kernel launch:

```
add<<< (N + M-1) / M, M >>> (d_a, d_b, d_c, N);
```

왜 스레드를 사용해야 하나요?

- 불필요해 보이는 스레드
 - 스레드는 복잡성을 더함.
 - 우리가 얻는 것은 무엇인가요?
- 병렬 블록과 달리 스레드에는 다음과 같은 메커니즘 있음:
 - 통신
 - 동기화
- 자세히 살펴보려면 새로운 예시가 필요합니다...

CUDA Thread Indexing



- 그리드에 있는 블록 수:
 - `gridDim.x` - 그리드의 x 차원에 있는 블록 수(예: 3)
 - `gridDim.y` - 그리드의 y 차원에 있는 블록 수(예: 2)
- 블록의 스레드 수:
 - `blockDim.x` - 블록의 x 차원에 있는 스레드 수(예: 4)
 - `blockDim.y` - 블록의 y 차원에 있는 스레드 수(예: 3)
- 블록 인덱스:
 - `blockIdx.x` - 블록의 x 차원 인덱스
 - `blockIdx.y` - y 차원에서의 블록 인덱스
 - 예: 블록 (0,1) - `blockIdx.x = 0` , `blockIdx.y = 1`
- 스레드 인덱스:
 - `threadIdx.x` - x 차원에서의 스레드 인덱스
 - `threadIdx.y` - y 차원에서의 스레드 인덱스
 - 예: Thread(2,1) - `threadIdx.x = 2` , `threadIdx.y = 1`

•그리드(grid)

- 커널(GPU 기능)은 그리드라는 스레드 블록의 모음으로 실행.

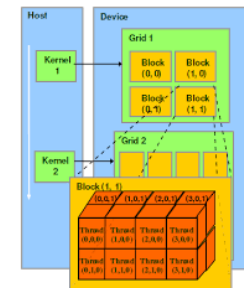
- 그리드는 스레드 블록으로 구성. 그리드 크기는 블록 수를 사용하여 정의. 예를 들어 크기 6의 그리드에는 6개의 스레드 블록이 포함되어 있음. 그리드가 1D인 경우 →6개의 블록이 모두 한 차원(예: 1x6)입니다. 그리드가 2D인 경우 →6개의 블록이 2 차원입니다(예: 3x2).

•블록(block)

- 스레드의 모음.

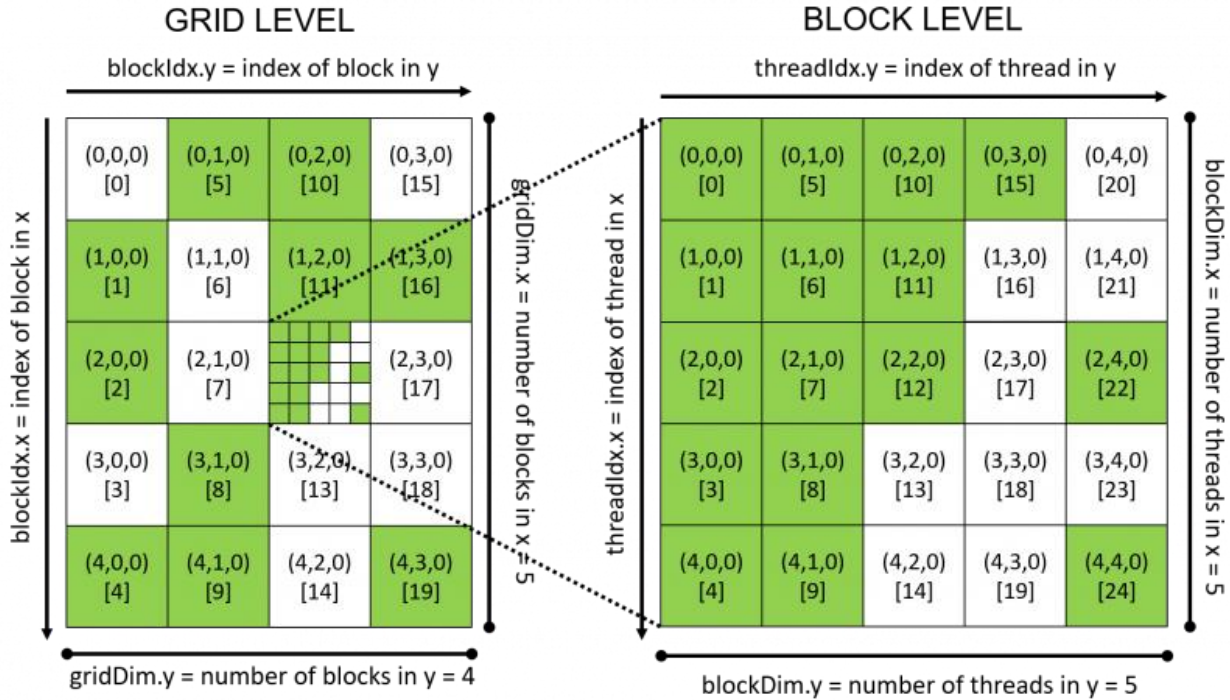
•스레드(thread)

- GPU에서 GPU function(커널)을 실행하는 단일 실행



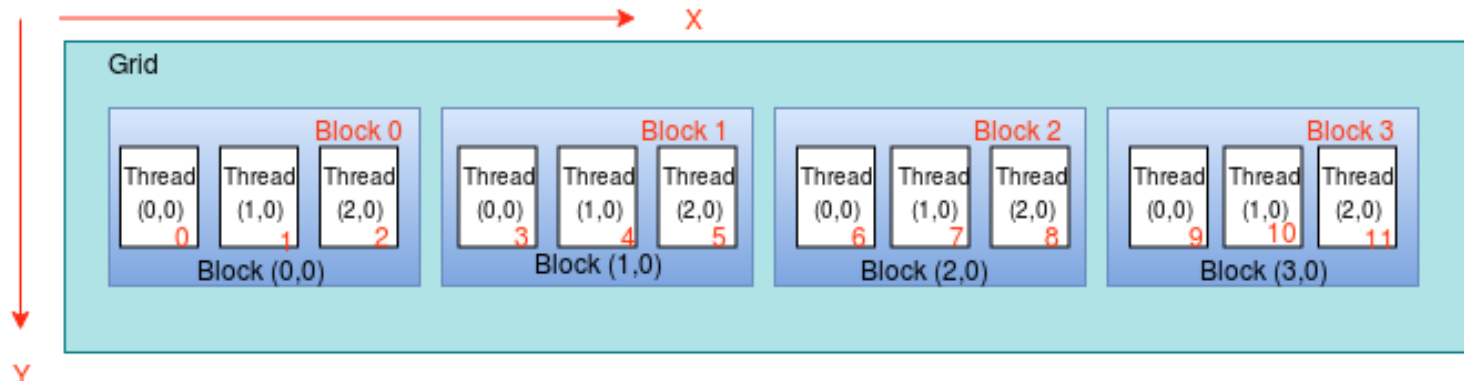
CUDA Thread Indexing

NAME	DESCRIPTION	x	y	z
<code>threadIdx</code>	Thread index within the block (zero-based)	<code>threadIdx.x</code>	<code>threadIdx.y</code>	<code>threadIdx.z</code>
<code>blockIdx</code>	Block index within the grid (zero-based)	<code>blockIdx.x</code>	<code>blockIdx.y</code>	<code>blockIdx.z</code>
<code>blockDim</code>	Block dimensions in threads	<code>blockDim.x</code>	<code>blockDim.y</code>	<code>blockDim.z</code>
<code>gridDim</code>	Grid dimensions in blocks	<code>gridDim.x</code>	<code>gridDim.y</code>	<code>gridDim.z</code>



CUDA Thread Indexing

1D grid of 1D blocks



빨간색으로 표시된 인덱스는 각 블록과 각 스레드의 고유 번호입니다.

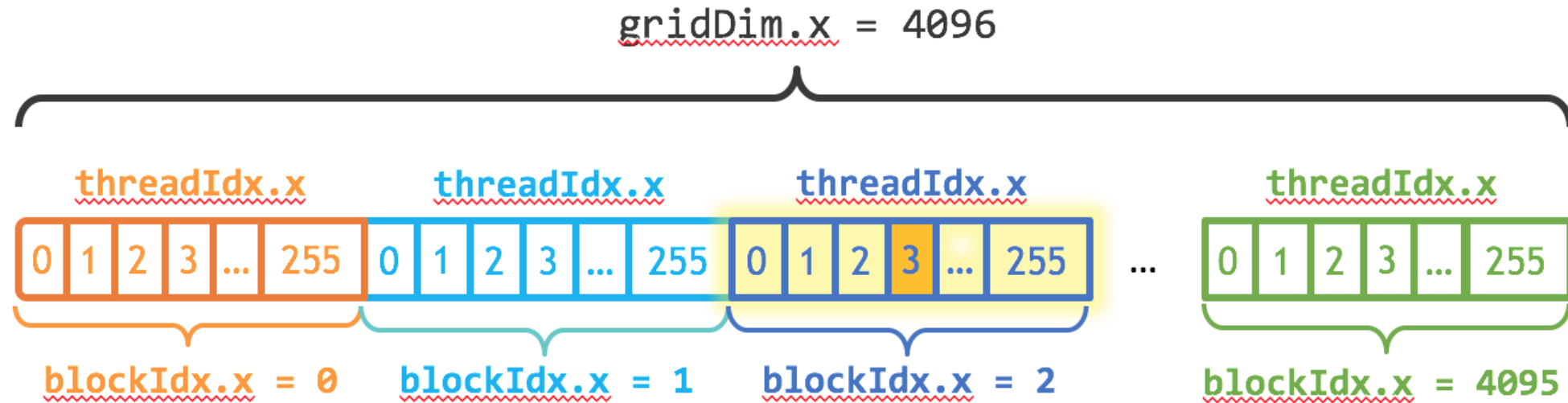
$$threadId = (blockIdx.x * blockDim.x) + threadIdx.x$$

블록(1,0)의 스레드(2,0)에 대한 방정식을 확인해 보겠습니다.

$$Thread\ ID = (1 * 3) + 2 = 3 + 2 = 5$$

CUDA Thread Indexing

1D grid of 1D blocks



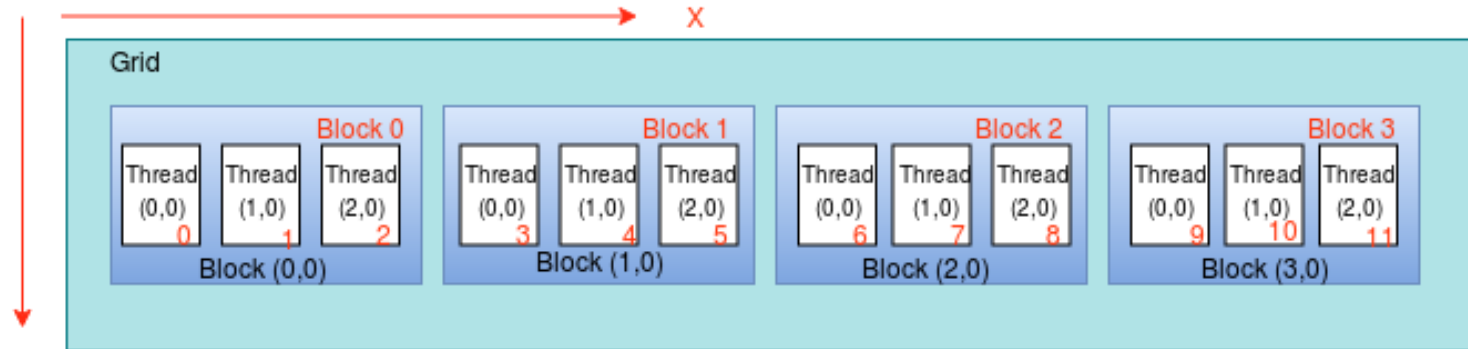
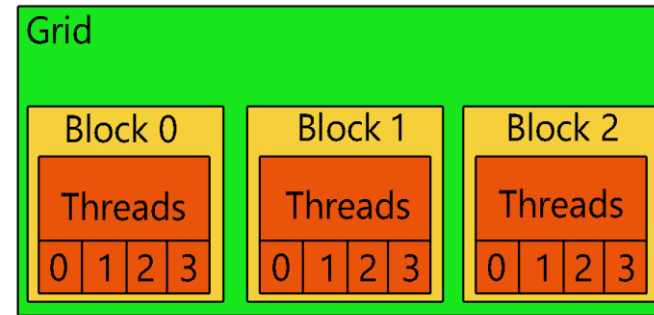
$$\text{index} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$

$$\text{index} = (2) * (256) + (3) = 515$$

CUDA Thread Indexing

1D grid of 1D blocks

1D Grid of 1D Blocks



빨간색으로 표시된 인덱스는 각 블록과 각 스레드의 고유 번호입니다.

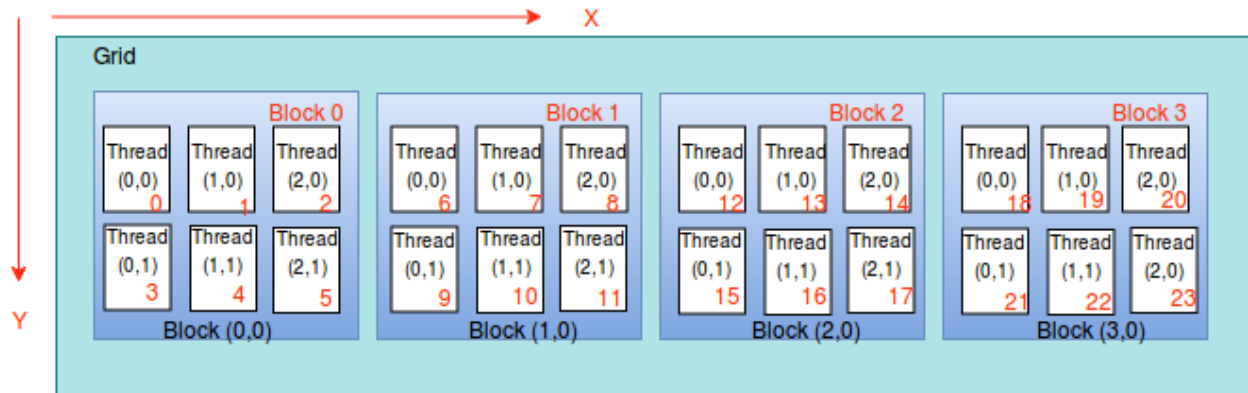
```
int threadId = (blockIdx.x * blockDim.x) + threadIdx.x
```

블록(1,0)의 스레드(2,0)에 대한 방정식을 확인해 보겠습니다.

$$\text{Thread ID} = (1 * 3) + 2 = 3 + 2 = 5$$

CUDA Thread Indexing

1D grid of 2D blocks (1)



빨간색으로 표시된 인덱스는 각 블록과 각 스레드의 고유 번호입니다.

$threadId = (blockIdx.x * blockDim.x * blockDim.y) + (threadIdx.y * blockDim.x) + threadIdx.x$
블록(1,0)의 스레드(2,1)에 대한 방정식을 확인해 보겠습니다.

$$\text{Thread ID} = (1 * 3 * 2) + (1 * 3) + 2 = 6 + 3 + 2 = 11$$

- 여기서 $(1 * 3 * 2) \rightarrow$ 블록 0의 스레드 수 계산
- $(1 * 3) \rightarrow$ 블록 1에서 스레드(0,0), (1,0), (2,0)을 계산
- 그런 다음 특정 스레드의 $threadIdx.x$ 를 추가

1D Grid of 2D Blocks

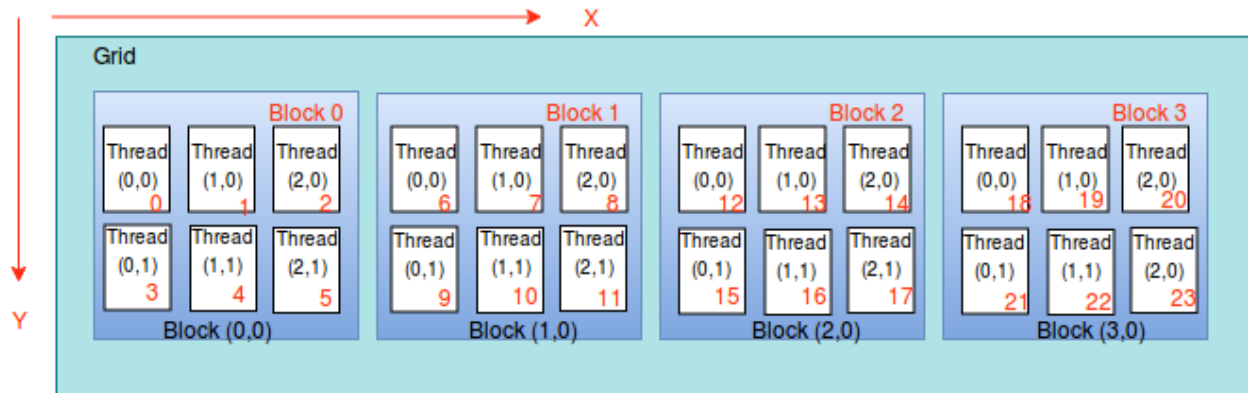
Grid				
Block 0				
Threads				
0,0	0,1	0,2	0,3	0,4
1,0	1,1	1,2	1,3	1,4
2,0	2,1	2,2	2,3	2,4
3,0	3,1	3,2	3,3	4,4

Block 1				
Threads				
0,0	0,1	0,2	0,3	0,4
1,0	1,1	1,2	1,3	1,4
2,0	2,1	2,2	2,3	2,4
3,0	3,1	3,2	3,3	4,4

Block 2				
Threads				
0,0	0,1	0,2	0,3	0,4
1,0	1,1	1,2	1,3	1,4
2,0	2,1	2,2	2,3	2,4
3,0	3,1	3,2	3,3	4,4

CUDA Thread Indexing

1D grid of 2D blocks (2)



빨간색으로 표시된 인덱스는 각 블록과 각 스레드의 고유 번호입니다.

$$threadId = (gridDim.x * blockDim.x * threadIdx.y) + (blockDim.x * blockIdx.x) + threadIdx.x$$

블록(1,0)의 스레드(2,1)에 대한 방정식을 확인해 보겠습니다.

$$\text{Thread ID} = (4 * 3 * 1) + (1 * 3) + 2 = 12 + 3 + 2 = 17$$

- 여기서 $(4 * 3 * 1) \rightarrow$ 블록 0,1,2,3에서 스레드(0,0),(1,0),(2,0)을 계산
- $(1 * 3) \rightarrow$ 블록 0에서 스레드(0,1),(1,1),(2,1)을 계산
- 그런 다음 특정 스레드의 $threadIdx.x$ 를 추가

CUDA Thread Indexing

2D grid of 1D blocks



빨간색으로 표시된 인덱스는 각 블록과 각 스레드의 고유 번호입니다.

$$blockId = (gridDim.x * blockIdx.y) + blockIdx.x$$

$$threadId = (blockId * blockDim.x) + threadIdx.x$$

블록(1,1)의 스레드(1,0)에 대한 방정식을 확인해 보겠습니다.

$$blockId = (2 * 1) + 1 = 2 + 1 = 3$$

$$threadId = (3 * 3) + 1 = 9 + 1 = 10$$

CUDA Thread Indexing

2D grid of 2D blocks



빨간색으로 표시된 인덱스는 각 블록과 각 스레드의 고유 번호입니다.

$$blockId = (gridDim.x * blockIdx.y) + blockIdx.x$$

$$threadId = (blockId * (blockDim.x * blockDim.y)) + (threadIdx.y * blockDim.x) + threadIdx.x$$

블록(0,1)의 스레드(2,1)에 대한 방정식을 확인해 보겠습니다.

$$blockId = (2 * 1) + 0 = 2$$

$$ThreadId = (2 * (3 * 2)) + (1 * 3) + 2 = 12 + 3 + 2 = 17$$

CUDA Thread Indexing

1D grid of 1D blocks

```
__device__
int getGlobalIdx_1D_1D(){
    return blockIdx.x * blockDim.x + threadIdx.x;
}
```

1D grid of 2D blocks

```
__device__
int getGlobalIdx_1D_2D(){
    return blockIdx.x * blockDim.x * blockDim.y
        + threadIdx.y * blockDim.x + threadIdx.x;
}
```

1D grid of 3D blocks

```
__device__
int getGlobalIdx_1D_3D(){
    return blockIdx.x * blockDim.x * blockDim.y * blockDim.z
        + threadIdx.z * blockDim.y * blockDim.x
        + threadIdx.y * blockDim.x + threadIdx.x;
}
```

2D grid of 1D blocks

```
__device__ int getGlobalIdx_2D_1D(){
    int blockId = blockIdx.y * gridDim.x + blockIdx.x;
    int threadId = blockId * blockDim.x + threadIdx.x;
    return threadId;
}
```

2D grid of 2D blocks

```
__device__
int getGlobalIdx_2D_2D(){
    int blockId = blockIdx.x + blockIdx.y * gridDim.x;
    int threadId = blockId * (blockDim.x * blockDim.y)
        + (threadIdx.y * blockDim.x) + threadIdx.x;
    return threadId;
}
```

2D grid of 3D blocks

```
__device__
int getGlobalIdx_2D_3D(){
    int blockId = blockIdx.x + blockIdx.y * gridDim.x;
    int threadId = blockId * (blockDim.x * blockDim.y * blockDim.z)
        + (threadIdx.z * (blockDim.x * blockDim.y))
        + (threadIdx.y * blockDim.x) + threadIdx.x;
    return threadId;
}
```

3D grid of 1D blocks

```
__device__
int getGlobalIdx_3D_1D(){
    int blockId = blockIdx.x + blockIdx.y * gridDim.x
        + gridDim.x * gridDim.y * blockIdx.z;
    int threadId = blockId * blockDim.x + threadIdx.x;
    return threadId;
}
```

3D grid of 2D blocks

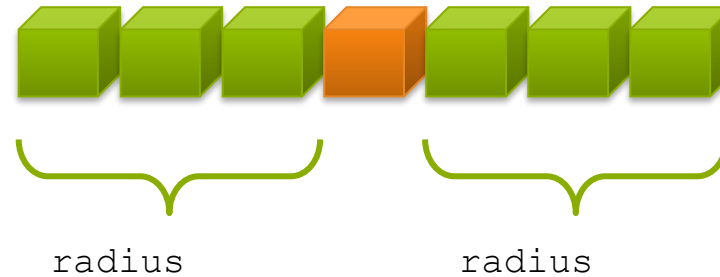
```
__device__
int getGlobalIdx_3D_2D(){
    int blockId = blockIdx.x + blockIdx.y * gridDim.x
        + gridDim.x * gridDim.y * blockIdx.z;
    int threadId = blockId * (blockDim.x * blockDim.y)
        + (threadIdx.y * blockDim.x) + threadIdx.x;
    return threadId;
}
```

3D grid of 3D blocks

```
__device__
int getGlobalIdx_3D_3D(){
    int blockId = blockIdx.x + blockIdx.y * gridDim.x
        + gridDim.x * gridDim.y * blockIdx.z;
    int threadId = blockId * (blockDim.x * blockDim.y * blockDim.z)
        + (threadIdx.z * (blockDim.x * blockDim.y))
        + (threadIdx.y * blockDim.x) + threadIdx.x;
    return threadId;
}
```

1D Stencil

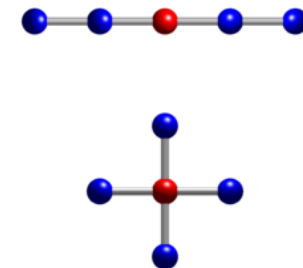
- 요소의 1D 배열에 1D 스텐실을 적용한다고 가정
 - 각 출력 요소는 반경 내의 입력 요소의 합계.
- 반경이 3이면 각 출력 요소는 7개의 입력 요소의 합:



- [수치 분석](#) 에서 1차원 또는 2차원의 [정사각형 그리드가](#) 주어지면 그리드에 있는 점의 **5점 스텐실**은 점 자체와 4개의 "이웃"으로 구성된 [스텐실](#) 입니다.
- 이는 그리드 점에서 [도함수](#) 에 대한 [유한 차분](#) 근사를 작성하는 데 사용됩니다 . [수치미분](#) 의 예이다 . 1차원에서 그리드의 점 사이의 간격이 h 이면 그리드의 점 x 의 5점 스텐실은 다음과 같습니다.

$$\{x - 2h, x - h, x, x + h, x + 2h\}.$$

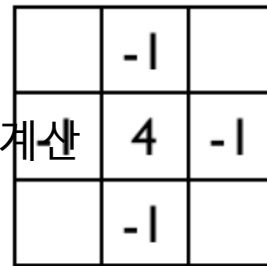
$$f'(x) \approx \frac{-f(x + 2h) + 8f(x + h) - 8f(x - h) + f(x - 2h)}{12h}$$



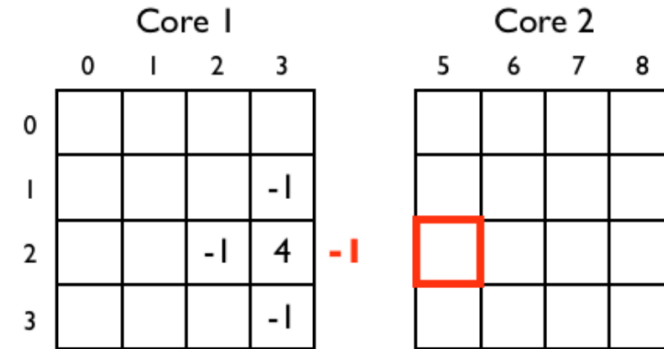
Stencil?

- 점의 계산에 영향을 주는 인접한 점의 집합을 흔히 스텐실이라고 함.
- 스텐실은 점의 값을 자체값과 이웃 점의 값에서 계산하는 방법을 정의.
- 스텐실은 다양한 형태를 취할 수 있으며 현재 점과 직접 인접하지 않은 점을 포함할 수 있음.

그림1) 기하학적으로 분해된 그리드에서의 스텐실 계산



(a) 5-Point Stencil



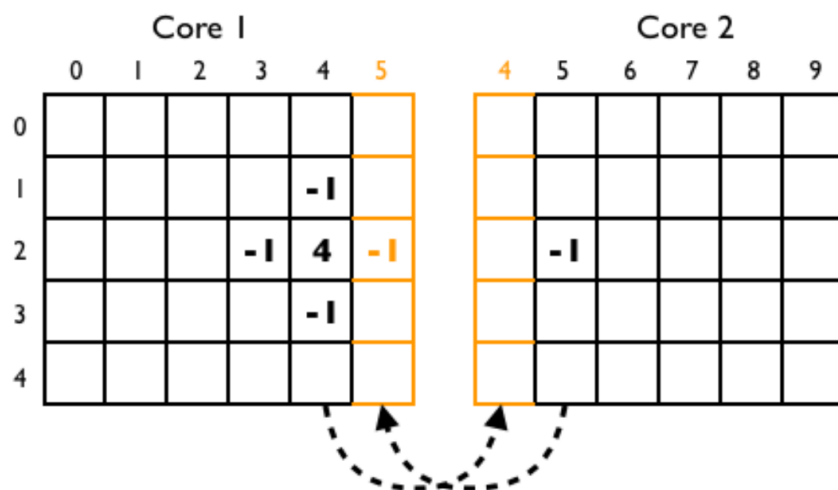
(b) Stencil that needs a cell from its neighbor

- 그림1(a)는 이미지에서 가장자리를 찾는 데 사용할 수 있는 스텐실인 5점 라플라스 연산자를 나타냄.
- 현재 반복에서 한 점의 값은 이전 반복의 왼쪽, 오른쪽, 위, 아래 이웃 점의 값에서 자신의 값을 뺀 값에 4를 곱한 값으로 계산된다.
- 기하학적 분해[1]는 서로 다른 프로세스 또는 스레드를 사용하여 이러한 그리드의 값을 병렬로 계산하는 일반적인 패턴.
- 이 패턴의 나머지 부분에서는 공유 메모리와 직접 관련된 문제를 다루지 않는 한 프로세스와 스레드를 모두 포괄하기 위해 프로세스라는 용어를 사용(공유메모리와 관련 있는 경우 스레드라는 용어를 사용).
- 기본 아이디어는 그리드를 청크(덩어리)로 나누고 각 프로세스가 이 중 하나 이상을 업데이트하도록 하는 것.
- 그림 1(b)에서 볼 수 있듯이 이 접근 방식의 일반적인 문제는 청크 사이의 경계에서 값을 계산하는 방법인데, 하나 이상의 인접한 청크의 값이 필요하기 때문.
- 필요에 따라 이웃 청크를 처리하는 프로세스에서 필요한 포인트를 가져오는 것은 계산 중간에 많은 작은 통신 작업이 발생하여 지연 시간이 길어지기 때문에 좋은 해결책이 아님.

Stencil?

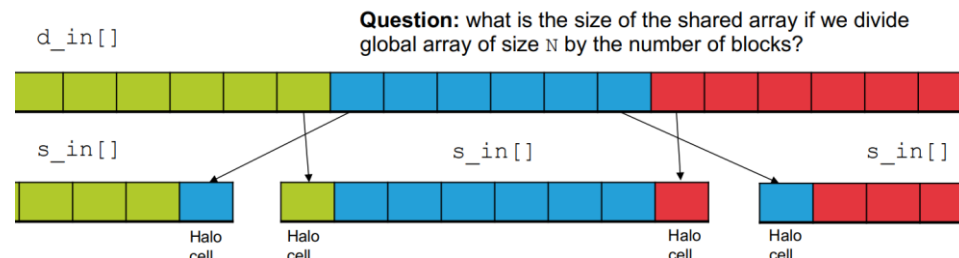
그림 2) 각 청크는 인접한 청크에서 고스트 셀(Halo) 벡터를 받음.

해결책



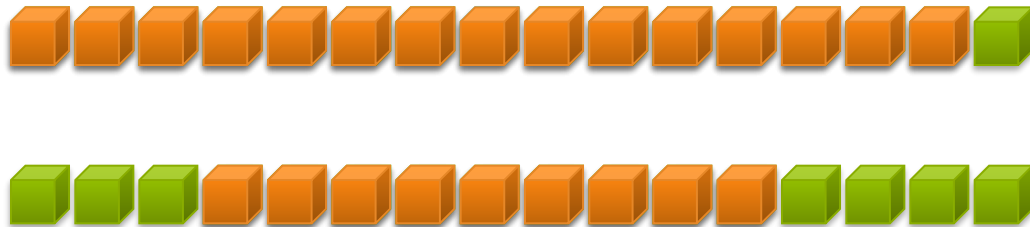
- 각 청크의 가장자리 주위에 일련의 고스트 셀을 위한 추가 공간을 할당. 반복할 때마다 각 쌍의 이웃이 테두리를 교환하고 그림 2와 같이 받은 테두리를 고스트 셀 영역에 배치함.
- 고스트 셀은 모든 인접한 이웃의 테두리 복제본을 포함하는 각 청크 주위에 헤일로 을 형성. 이러한 고스트 이미지는 로컬로 업데이트되지 않지만 이 청크의 테두리를 업데이트할 때 스텐실 값을 제공.

전역 배열을 같은 크기의 작은 배열로 나눕니다. 문제: 타일의 경계 셀에서 $(u_{i+1} - 2u_i + u_{i-1})/h^2$ 를 계산하려면 인접한 부분의 값을 알아야 함.
해결 방법: 공유 배열에 인접한 부분의 경계에 배열 요소를 포함하세요.



블록 내에서 구현하기

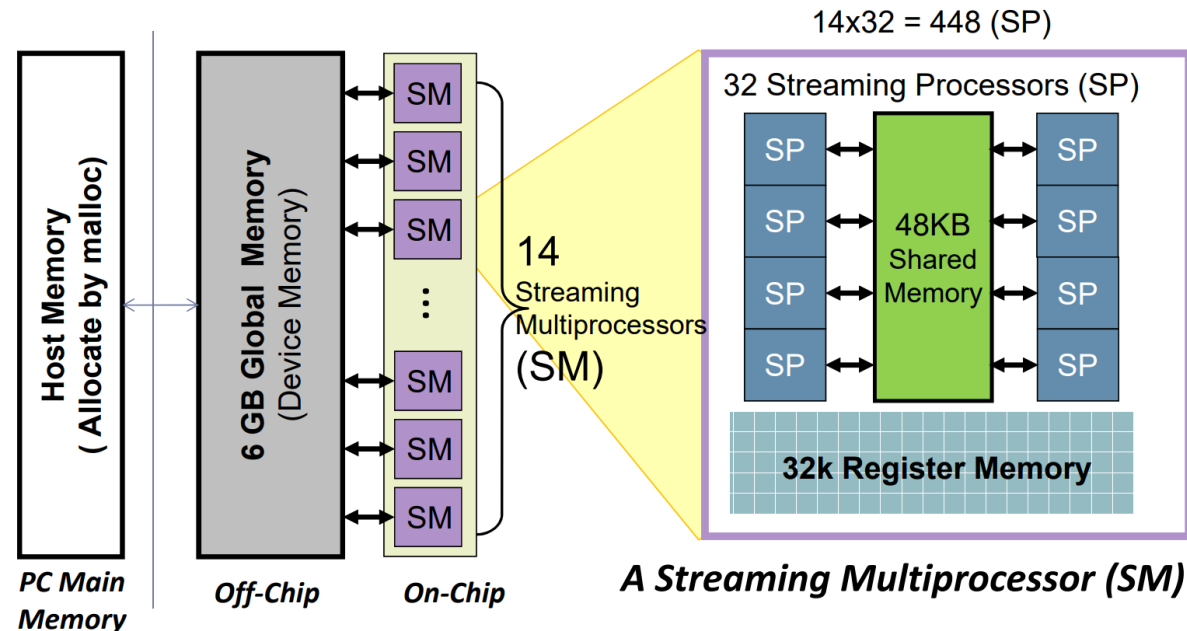
- 각 스레드는 하나의 출력 요소를 처리
blockDim.x elements per block
- 입력 요소는 여러 번 읽음
 - 반경 3을 사용하면 각 입력 요소를 7번 읽음.



스레드 간 데이터 공유

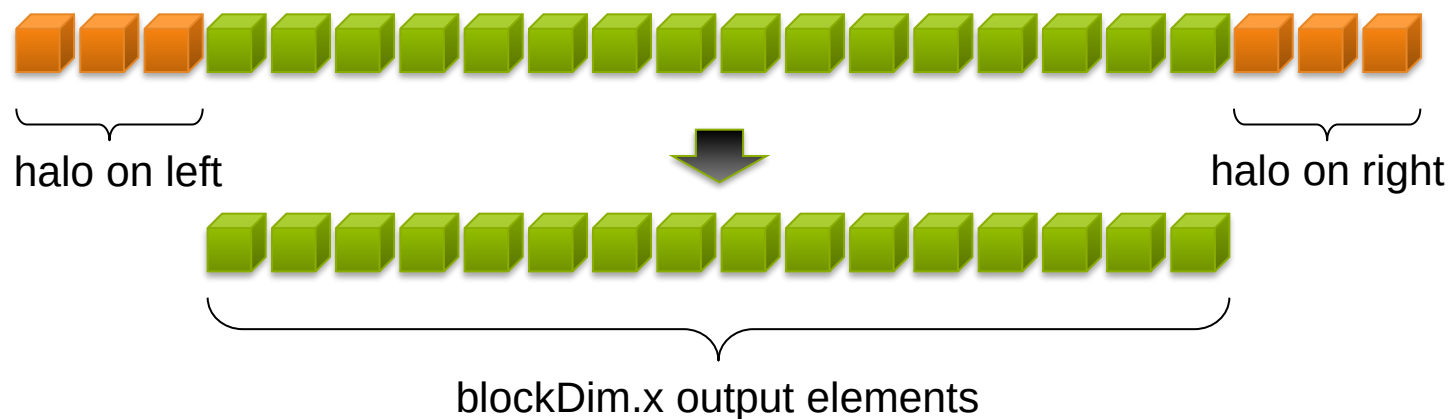
- Terminology: 블록 내에서 스레드는 공유(shared memory) 메모리를 통해 데이터를 공유
 - ✓매우 빠른 온칩 메모리, 사용자 관리형
 - ✓블록당 할당된 `_shared_`를 사용하여 선언
 - ✓다른 블록의 스레드에는 데이터가 보이지 않음(참조 할 수 없음)

▶ Highly parallel, multithreaded, many-core processor



Implementing With Shared Memory

- 공유 메모리에 데이터 캐시
 - 전역 메모리에서 공유 메모리로 입력 요소 읽기($\text{blockDim.x} + 2 * \text{radius}$)
 - blockDim.x 출력 요소 계산
 - blockDim.x 출력 요소를 전역 메모리에 쓰기
 - 각 블록은 각 경계에 반경 요소의 halo가 필요.



Stencil Kernel

```
__global__ void stencil_1d(int *in, int *out) {  
    __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];  
    int gindex = threadIdx.x + blockIdx.x * blockDim.x;  
    int lindex = threadIdx.x + RADIUS;
```



```
    // Read input elements into shared memory
```

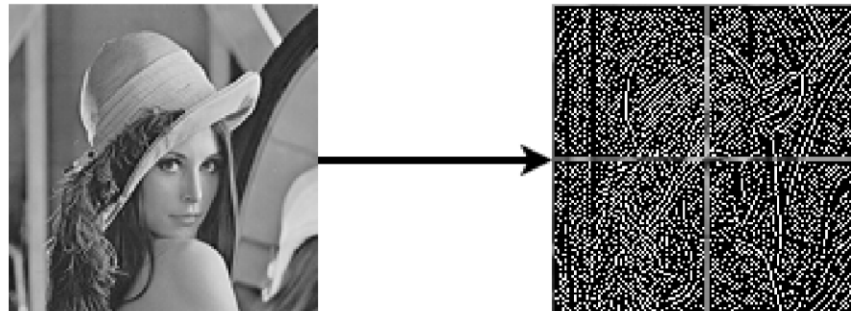
```
    temp[lindex] = in[gindex];  
    if (threadIdx.x < RADIUS) {  
        temp[lindex - RADIUS] = in[gindex - RADIUS];  
        temp[lindex + BLOCK_SIZE] =  
            in[gindex + BLOCK_SIZE];  
    }
```



Stencil Kernel

```
// Apply the stencil  
int result = 0;  
for (int offset = -RADIUS ; offset <= RADIUS ; offset++)  
    result += temp[lindex + offset];  
  
// Store the result  
out[gindex] = result;  
}
```

스텐실 형태의 계산문제가 많이 존재(영상처리, Convolution)



Multiprocess edge detection without border exchanges

Data Race!

- 스텐실 예제가 작동하지 않습니다...
- 스레드 0이 halo을 가져오기 전에 스레드 15가 halo을 읽었다고 가정...

```
temp[lindex] = in[gindex];           Store at temp[18]   
if (threadIdx.x < RADIUS) {  
    temp[lindex - RADIUS] = in[gindex - RADIUS];    Skipped, threadIdx > RADIUS  
    temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];  
}  
  
int result = 0;  
result += temp[lindex + 1];          Load from temp[19] 
```


__syncthreads()

- `void __syncthreads ();`
- 블록 내의 모든 스레드를 동기화.
 - RAW/WAR/WAW 위험을 방지하는 데 사용.
- 모든 스레드가 배리어에 도달해야 함.
 - 조건부 코드에서는 조건이 블록 전체에 걸쳐 균일해야 함.

Stencil Kernel

```
__global__ void stencil_1d(int *in, int *out) {  
    __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];  
    int gindex = threadIdx.x + blockIdx.x * blockDim.x;  
    int lindex = threadIdx.x + radius;  
  
    // Read input elements into shared memory  
    temp[lindex] = in[gindex];  
    if (threadIdx.x < RADIUS) {  
        temp[lindex - RADIUS] = in[gindex - RADIUS];  
        temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];  
    }  
  
    // Synchronize (ensure all the data is available)  
    __syncthreads();  
}
```

Stencil Kernel

```
// Apply the stencil  
int result = 0;  
for (int offset = -RADIUS ; offset <= RADIUS ; offset++)  
    result += temp[lindex + offset];  
  
// Store the result  
out[gindex] = result;  
}
```

- 병렬 스레드 시작하기

- `kernel<<<N,M>>> (...);` 을 사용하여 블록당 M개의 스레드로 N개의 블록을 실행
- `blockIdx.x` 를 사용하여 그리드 내 블록 인덱스에 액세스하기
- `threadIdx.x` 를 사용하여 블록 내 스레드 인덱스 액세스

- 요소를 스레드에 할당

```
int index = threadIdx.x + blockIdx.x * blockDim.x
```

- 공유 메모리에 변수/배열을 선언하려면 `__shared__`를 사용.
 - 블록의 스레드 간에 데이터가 공유.
 - 다른 블록의 스레드에는 볼 수 없음
- barrier(동기화)을 위해 `__syncthreads()` 사용
 - 데이터 위험(data hazards)방지를 위해 사용

- Kernel launches(실행)는 **asynchronous(비동기)**이다
 - 제어권은 즉시 CPU로 넘어감
- CPU는 결과를 소비하기 전에 동기화해야 합니다.

`cudaMemcpy()`

- 복사가 완료될 때까지 CPU를 차단(Block).
- 선행하는 모든 CUDA 호출이 완료되면 복사를 시작.

`cudaMemcpyAsync()`

- Asynchronous, block the CPU하지 않음

`cudaDeviceSynchronize()`

- 모든 선행 CUDA 호출이 완료될 때까지 CPU를 블록함

Reporting Errors

- All CUDA API calls return an error code (`cudaError_t`)
 - Error in the API call itself
 - OR
 - Error in an earlier asynchronous operation (e.g. kernel)

- Get the error code for the last error:

```
cudaError_t cudaGetLastError(void)
```

- Get a string to describe the error:

```
char *cudaGetErrorString(cudaError_t)
```

```
printf("%s\n", cudaGetErrorString(cudaGetLastError()));
```

Device Management

- Application can query and select GPUs

```
cudaGetDeviceCount(int *count)
cudaSetDevice(int device)
cudaGetDevice(int *device)
cudaGetDeviceProperties(cudaDeviceProp *prop, int device)
```

- Multiple threads can share a device
- A single thread can manage multiple devices

```
cudaSetDevice(i) to select current device
cudaMemcpy(...) for peer-to-peer copies†
```

[†] requires OS and device support

Compute Capability(계산능력)

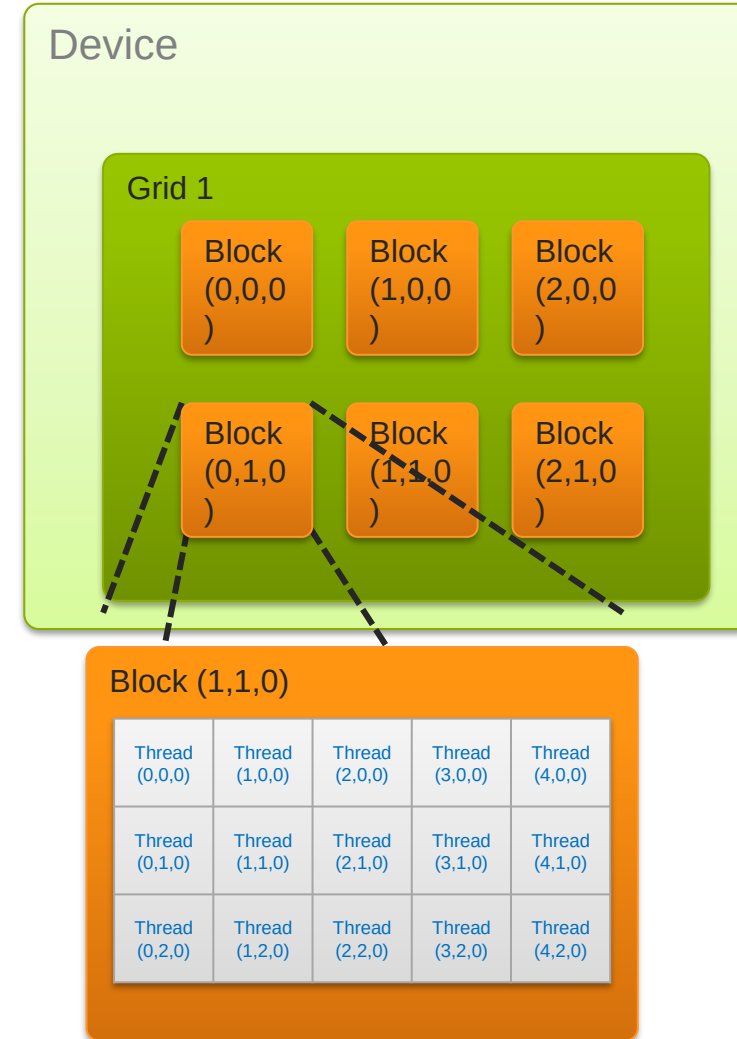
- The **compute capability** of a device describes its architecture, e.g.
 - Number of registers
 - Sizes of memories
 - Features & capabilities

Compute C apability	Selected Features (see CUDA C Programming Guide for complete list)	Tesla models
1.0	Fundamental CUDA support	870
1.3	Double precision, improved memory accesses, atomics	10-series
2.0	Caches, fused multiply-add, 3D grids, surfaces, ECC, P2 P, concurrent kernels/copies, function pointers, recursion	20-series

- The following presentations concentrate on Fermi devices
 - Compute Capability ≥ 2.0

IDs and Dimensions

- A kernel는 grid의 blocks 의 threads로 실행
 - blockIdx 와 threadIdx 는 3D
 - one dimension (x) 만 보임
- 내장(Built-in)변수
 - threadIdx
 - blockIdx
 - blockDim
 - gridDim



- **NVIDIA GPU에서 GPU 컴퓨팅을 가능하게 하는 컴파일러, 라이브러리 등을 포함한 프레임워크**
- **host (CPU) code와 GPU kernel 함수를 기술하는 device code로 구성된다.**
- **host code에는 CUDA API(그에 따른 변수)와 GPU kernel function call이 포함된다.**
- **device code (GPU kernel 함수)는 thread의 실행 내용을 기술하며, thread ID 등의 내장 변수, 준비된 특수 함수 등이 포함되는 것 외에는 일반적인 C 언어를 사용할 수 있으며, return 값은 사용할 수 없다.**

Hello to CUDA

<https://cuda-tutorial.readthedocs.io/en/latest/tutorials/tutorial01/#acknowledgments>

A quick comparison between CUDA and C

C

```
void c_hello(){  
    printf("Hello World!\n");  
}  
  
int main() {  
    c_hello();  
    return 0;  
}
```

CUDA

```
__global__ void cuda_hello(){  
    printf("Hello World from GPU!\n");  
}  
  
int main() {  
    cuda_hello<<<1,1>>>();  
    return 0;  
}
```

vector addition.(in C)

<https://cuda-tutorial.readthedocs.io/en/latest/tutorials/tutorial01/#acknowledgments>

```
#define N 10000000

void vector_add(float *out, float *a, float *b, int n) {
    for(int i = 0; i < n; i++){
        out[i] = a[i] + b[i];
    }
}

int main(){
    float *a, *b, *out;

    // Allocate memory
    a = (float*)malloc(sizeof(float) * N);
    b = (float*)malloc(sizeof(float) * N);
    out = (float*)malloc(sizeof(float) * N);

    // Initialize array
    for(int i = 0; i < N; i++){
        a[i] = 1.0f; b[i] = 2.0f;
    }

    // Main function
    vector_add(out, a, b, N);
}
```

Converting vector addition to CUDA

<https://cuda-tutorial.readthedocs.io/en/latest/tutorials/tutorial01/#acknowledgments>

1. Copy `vector_add.c` to `vector_add.cu`

```
$> cp vector_add.c vector_add.cu
```

1. Convert `vector_add()` to GPU kernel

```
__global__ void vector_add(float *out, float *a, float *b, int n) {  
    for(int i = 0; i < n; i++){  
        out[i] = a[i] + b[i];  
    }  
}
```

1. Change `vector_add()` call in `main()` to kernel call

```
vector_add<<<1,1>>>>(out, a, b, N);
```

1. Compile and run the program

```
$> nvcc vector_add.c -o vector_add  
$> ./vector_add
```

Converting vector addition to CUDA

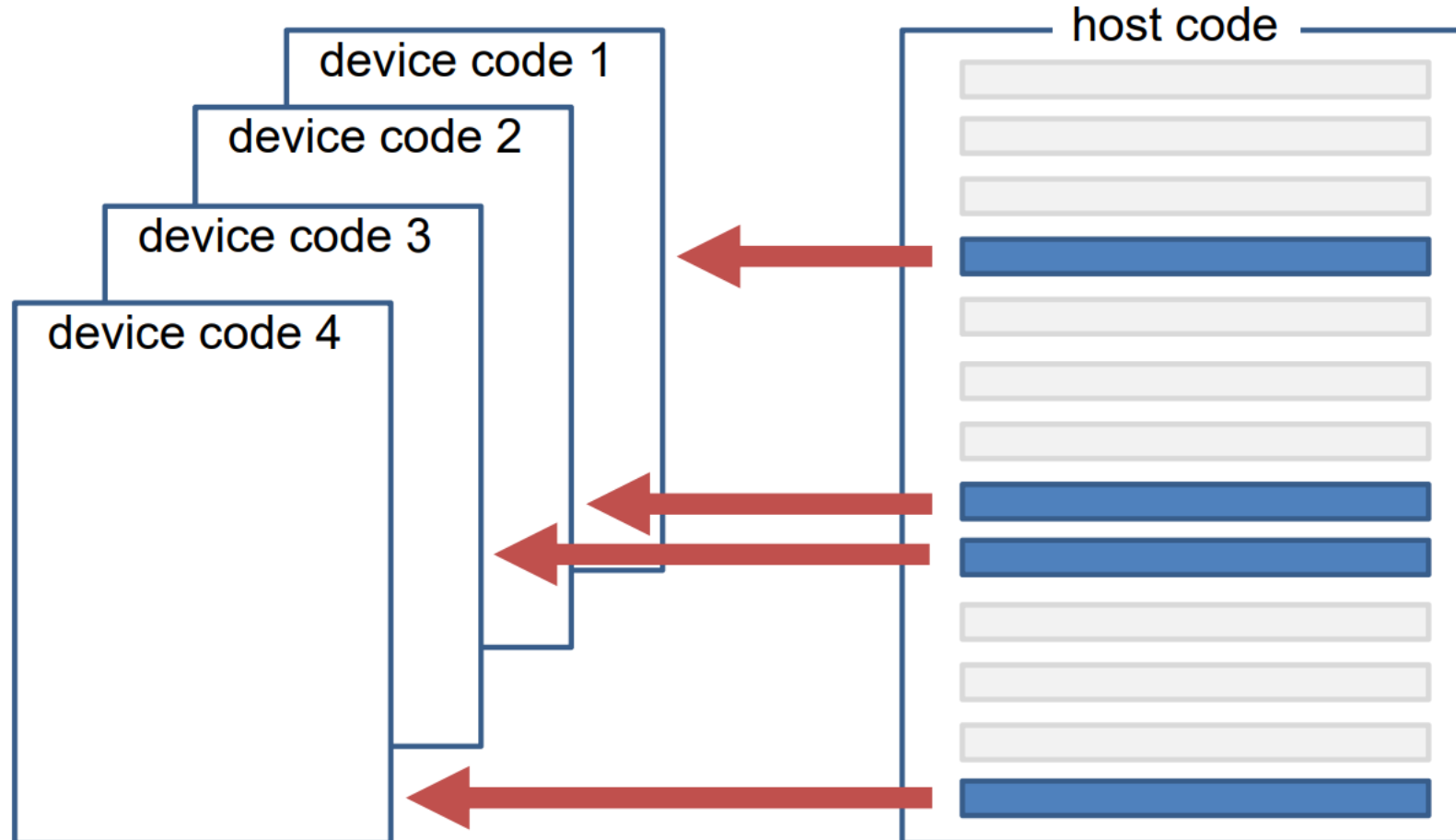
<https://github.com/PacktPublishing/Learn-CUDA-Programming>

<https://github.com/CUDA-Tutorial/CodeSamples>

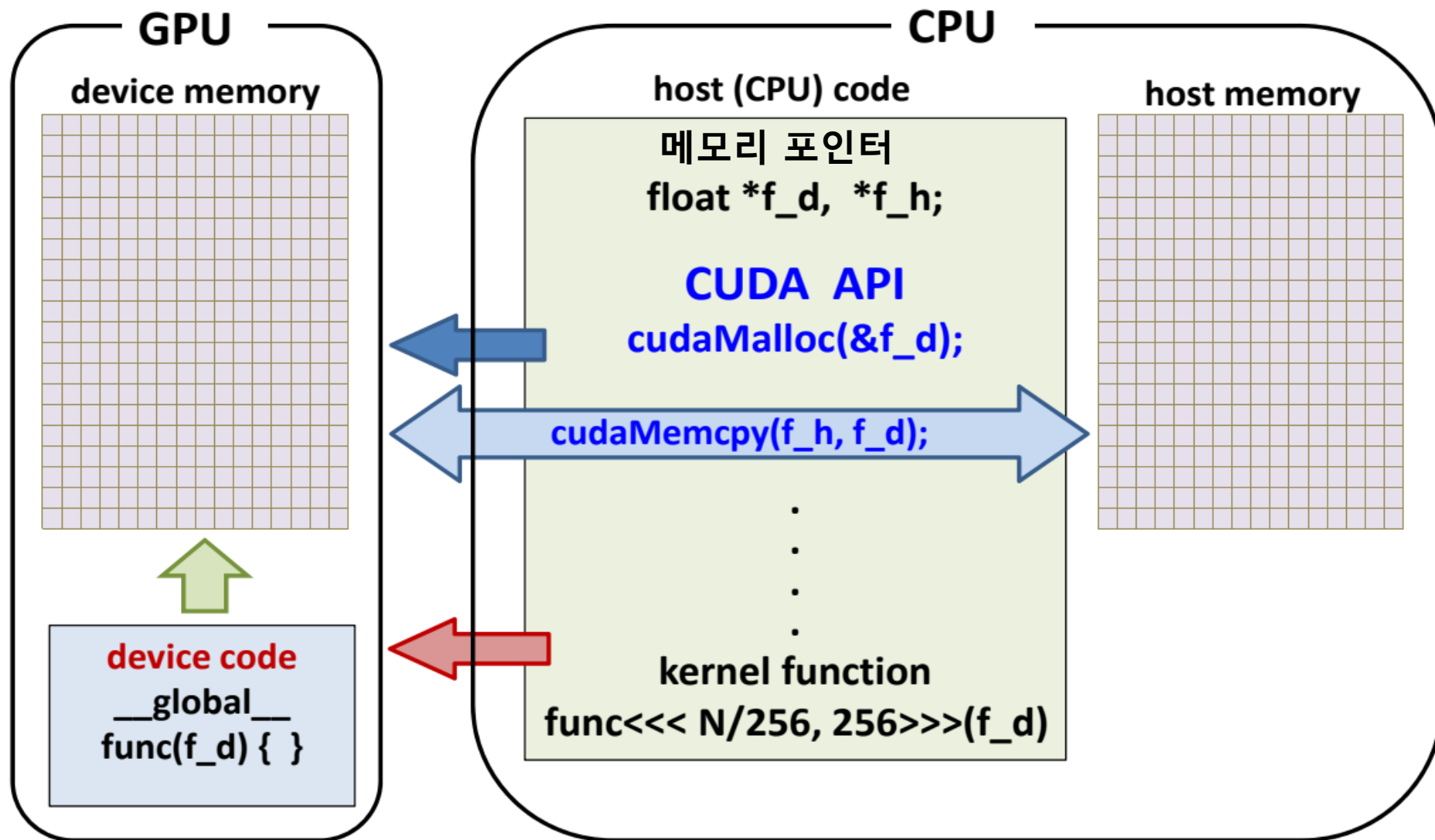
<https://cuda-tutorial.github.io/>

host code 와 device code

- GPU는 단독으로 동작하지 않으며, host를 CPU에서 실행하고 그 안에서 CUDA API와 GPU kernel 함수를 호출하는 방식으로 동작.



CUDA 프로그램 실행 개념도



CUDA 프로그램 실행 개념도

1. CUDA 소스 파일은 .cu 확장자를 붙인다.

- ① CUDA Toolkit의 nvcc로 컴파일한다.
- ② nvcc는 CPU에서 실행하는 코드와 GPU에서 실행하는 GPU kernel 함수 코드, CUDA의 API 부분을 분리한다.
- ③ CPU에서 실행하는 코드는 gcc, g++에 컴파일을 맡긴다.
- ④ GPU에서 실행하는 GPU 커널 함수 부분을 GPU용으로 컴파일한다.
- ⑤ GPU용 PTX 코드도 생성한다.

2. Library를 링크하여 실행 파일을 생성한다.

CUDA core library (cuda) -lcuda
CUDA runtime library (cudart) -lcudart

● 중요한 컴파일 옵션

[CUDA - Wikipedia](#)

[CUDA GPUs - Compute Capability | NVIDIA Developer](#)

<code>-arch sm_20</code>	• Compute Capability에 따라 컴파일한다. • DeviceQuery로 확인하고 그 이하를 지정한다.
<code>--maxrregcount <N></code>	• 하나의 kernel 함수당 사용할 레지스터 개수를 <N>으로 제한한다. 이를 통해, 지정한 병렬수로 thread를 실행할 수 있지만, 초과된 부분은 local 메모리에 놓여지게 되어 실행 속도가 느려진다. • <code>--maxrregcount 60</code>
<code>-use_fast_math</code>	• 빠른 수학 함수를 사용한다.
<code>-G</code>	• device 코드에 대해 디버깅을 가능하게 한다.
<code>--ptxas-options=-v</code>	• 레지스터와 메모리 사용 현황을 표시한다.

- Compute Capability는 NVIDIA의 CUDA 플랫폼에서 GPU의 기능 및 아키텍처 버전을 나타내는 지표
- 이 값에 따라 특정 GPU가 지원하는 CUDA가 결정.
- Compute Capability는 7.x, 8.x와 같이 버전별로 구분되어 있음.
- [CUDA GPUs - Compute Capability | NVIDIA Developer](#)

● 중요한 컴파일 옵션

```
$ nvcc --generate-code arch=compute_30,code=sm_30
```

- nvcc 컴파일러에 --generate-code 옵션을 지정하여 특정 환경에 맞는 (아마도 최적의) 코드를 생성하도록 지시할 수 있다.
- arch와 code의 하위 옵션에 대상 CC(Compute Capability)의 숫자를 지정한다. 사용 중인 NVIDIA GPU의 CC는 [CUDA GPUs | NVIDIA Developer](#)에서 확인할 수 있다.
- [CUDA GPUs - Compute Capability | NVIDIA Developer](#)
- 특정 CC용으로 생성된 PTX 코드는 동일하거나 더 새로운 CC용 바이너리로 컴파일할 수 있다. 예를 들어 CC5.0 (compute_50) 용으로 빌드한 PTX 코드는 CC6.0 환경에서도 (최적은 아니지만) 사용할 수 있지만, CC4.0 환경에서는 사용할 수 없다.
- 여러 CC에 대해 최적화하고 싶다면 여러 개를 지정한다. 그만큼 컴파일 시간이 늘어난다.

```
$ nvcc --generate-code arch=compute_30,code=sm_30 --generate-code arch=compute_50,code=sm_50
```

CUDA Memory 확보(1/2)

- 메모리 포인터는 device (GPU) memory에도 host (CPU) memory에도 사용할 수 있음.

예시) 단정밀도 실수: `float *f_d, *f_h;`

- device 상에 메모리를 확보하는 runtime API

`cudaMalloc(void **devptr, size_t count);`

- `devptr`: 디바이스 메모리 주소에 대한 포인터. 확보한 메모리 주소가 기록된다.
- `count`: 영역의 크기

예시) `cudaMalloc((void **)&f_d, sizeof(float)*n);`

`f_d[n]` 배열이 GPU 메모리에 할당된다.

CUDA Memory 확보(2/2)

float *f_h;

- **host 측에 pinned 메모리를 확보한다.**

cudaMallocHost(void **devptr, size_t count);



예시) cudaMallocHost((void **)&f_h, sizeof(float)*n);

- **f_h[n] 배열이 Host 메모리에 page lock (pinned)으로 확보된다.**
- **통상 pageable 메모리로 확보된 경우보다 전송 속도가 빠르다.**
- **또한, 비동기 통신의 경우에도 page lock 메모리에 한정된다.**

f_h = (float *) malloc(sizeof(float)*n); (통상)

CUDA 데이터 전송

```
float *f_d, *f_h;
```

```
cudaMemcpy(void *dst, const void *src, size_t count, enum cudaMemcpyKind kind)
```

dst: 전송 대상 메모리 주소

src: 전송할 메모리 주소

count: 영역의 크기

kind: 전송 유형을 지정하는 상수

`cudaMemcpyHostToDevice`

`cudaMemcpyDeviceToHost`

`cudaMemcpyDeviceToDevice`

예시) `cudaMemcpy (f_d, f_h, sizeof(float)*n, cudaMemcpyHostToDevice);`

host의 `f_h[n]` 배열의 데이터를 device의 `f_d[n]`에 복사한다.

GPU kernel-function call

host code 안에서 다음과 같이 호출한다.

```
kernel_function<<< Dg, Db, Ns, S>>>(a, b, c, ...);
```

- Dg: dim3 타입의 grid 크기 지정(Grid내 블록의 개수 지정)
- Db: dim3 타입의 block 크기 지정(블록내 스레드의 개수 지정)
- Ns: 실행 시 지정할 shared 메모리의 크기, 생략 가능: 생략할 경우 0으로 설정
- S: 비동기 실행의 stream 번호, 생략 가능: 생략 시 0이 설정된다, GPU의 thread 간에는 동기 실행이 된다.

Dg, Db로 지정한 수의 thread 가 실행된다.

kernel function의 실행은 CPU에 대해 끊임없이 비동기적으로 실행된다.

dim3 선언

kernel_function<<< **Dg, Db, Ns, S**>>>(a, b, c, ...);

의 Dg, Db를 **dim3**으로 지정한다.

dim3 a;	← 등가 →	dim3 a(1,1,1);
dim3 a(n, m);	← 등가 →	dim3 a(n, m, 1);
dim3 a(n, m, k);	← 등가 →	a.x = n; a.y = m; a.z = k;

dim3 a(n0, m0, k0); 은 선언과 함께 값 대입이다,
수시로 **a.x = n1; a.y = m1; a.z = k1;** 으로 변경 가능하다.

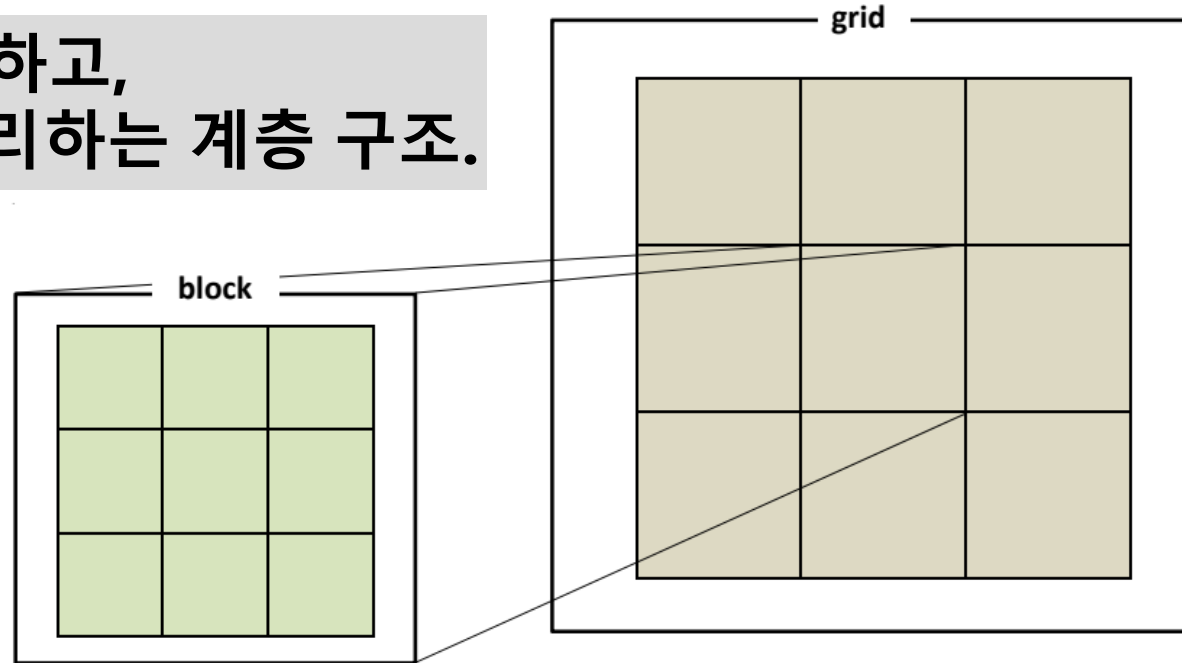
```
dim3 block (blocksize, 1, 1);  
dim3 grid  (N / block.x, 1, 1);
```

```
dim3 dimBlock(256);
```

Thread 관리

kernel 함수 <<< 제1인수, 제2인수 >>>로 지정
grid block

grid가 block을 관리하고,
block이 thread를 관리하는 계층 구조.



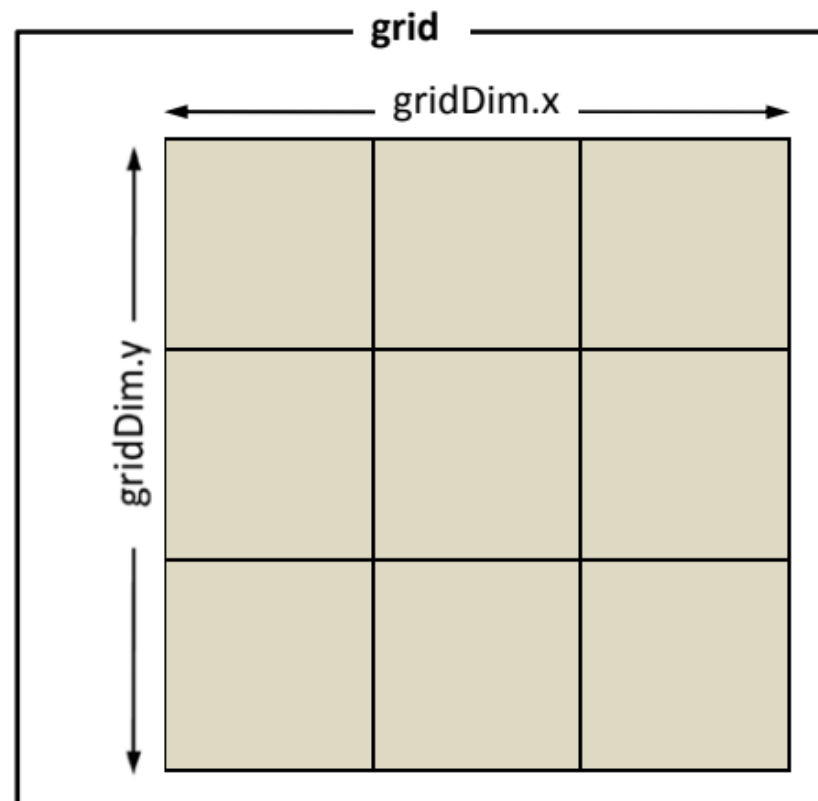
Thread 관리(grid)

kernel 함수의 첫 번째 인수를 다음과 같이 **grid**로 지정

```
dim3 grid(m, n, k);  
    grid.x = m;  
    grid.y = n;  
    grid.z = k;
```

예) grid 안에 $n*m$ 개의 Block이 있음.
kernel 함수 안에서는,

m → **gridDim.x**
n → **gridDim.y**



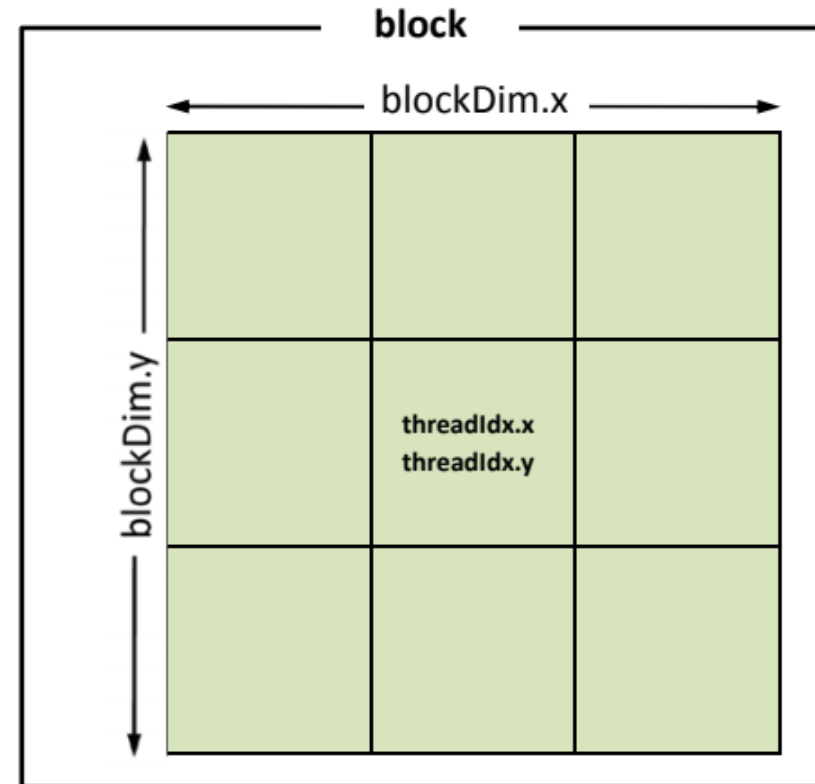
Thread 관리(block)

kernel 함수의 **두 번째 인수**를 다음과 같이 **block**로 지정

```
dim3 block(m, n, k);  
block.x = m;  
block.y = n;  
block.z = k;
```

block 안에 $n*m*k$ 개의 thread가 동작.
kernel 함수 안에서는 **blockIdx.x**, **blockIdx.y**,
blockIdx.z 로 지정할 수 있다.

$m \rightarrow \text{blockDim.x}$
 $n \rightarrow \text{blockDim.y}$
 $k \rightarrow \text{blockDim.z}$



Thread 관리(block)

- block 내의 Thread는 동일한 SM(Multi Processor)에서 실행됨.
- 동일 block 내 thread의 최대값은 1024이며, MP 내에는 32개의 SP만 있음에도 불구하고 64개 이상의 Thread 수로 실행하는 것이 효율적이다(성능은 더 좋을 것, 아마도.).
- 동일 block 내에서 실행되는 threads를 동기화할 수 있음.
(`__syncthreads();`)
- 동일 block 내의 threads는 shared 메모리를 공유 가능.
- 블록 내 thread 수를 늘리면 사용할 수 있는 레지스터 수가 줄어듦. (Fermi 의 경우, 256 thread 병렬 → 레지스터 수 32768/256)

Exercise

sample01.cu

```
#include <stdio.h>

int main(void) {
    printf("GPU compting¥n");
    return 0;
}
```

```
$ nvcc sample01.cu    // nvcc로 컴파일하기
$ ./a.out             // 실행
```

가장 간단한 커널 함수

sample01.cu

```
#include <stdio.h>
#include <cuda.h>
__global__ void built_in(void)
{
    printf("threadIdx.x=%d\n", threadIdx.x);
}
int main(void) {
    printf("GPU compting\n");
    built_in<<<1,3>>>();
    return 0;
}
```

```
$ nvcc -arch sm_20 sample01.cu
$ ./a.out
```

가장 간단한 커널 함수

sample01.cu

GPU 커널 함수 중 `printf()` 등을 사용하기 위해서는 `nvcc`로 컴파일할 때, 옵션 **-arch sm_20** (GPU capability에 따라 그 이상) 옵션이 필요하다.

`$ ./a.out`

를 실행해도 예상대로 표준 출력에 문자열이 표시되지 않는다.

왜 그럴까?

예제

CPU 측 배열 $a_h[n]$ 의 내용을 GPU의 global memory의 $a_d[n]$ 로 전송하고, GPU 커널에서 $a_d[n] \rightarrow b_d[n]$ 의 복사를 수행하며, 마지막으로 $b_d[n]$ 에서 CPU 측 배열 $b_h[n]$ 로 전송한다.

$a_h \rightarrow a_d \rightarrow b_d \rightarrow b_h$
(CPU) (GPU) (GPU) (CPU)

```
#define NUM (1024*1024*1) //1~16 등으로 변경 해보자
```

```
int n= NUM;  
double *a_h, *b_h, *a_d, *b_d;
```

```
a_h = (double *) malloc(n*sizeof(double)); for(i= 0; i < n; i++) a_h[i] = 9.3;  
b_h = (double *) malloc(n*sizeof(double)); for(i= 0; i < n; i++) b_h[i] = 0.0;
```

} host 측에
메모리 확보
와 대입

```
cudaMalloc( (void**) &a_d, n*sizeof(double) );  
cudaMalloc( (void**) &b_d, n*sizeof(double) );
```

} device 측에
메모리 확보

```
cudaMemcpy( Ad, A, n*sizeof(double), cudaMemcpyHostToDevice );  
cudaMemcpy( Bd, B, n*sizeof(double), cudaMemcpyHostToDevice );
```

} Host쪽에서 device 측에
데이터 전송

DATA 전송 테스트

`dim3 grid(n/256), block(256);` ← block size을 256으로 함
`g2g_copy<<< grid, block>>>(a_d, b_d);` ← kernel 함수 호출 실행
`//또는 g2g_copy<<< n/256, 256>>>(a_d, b_d);`

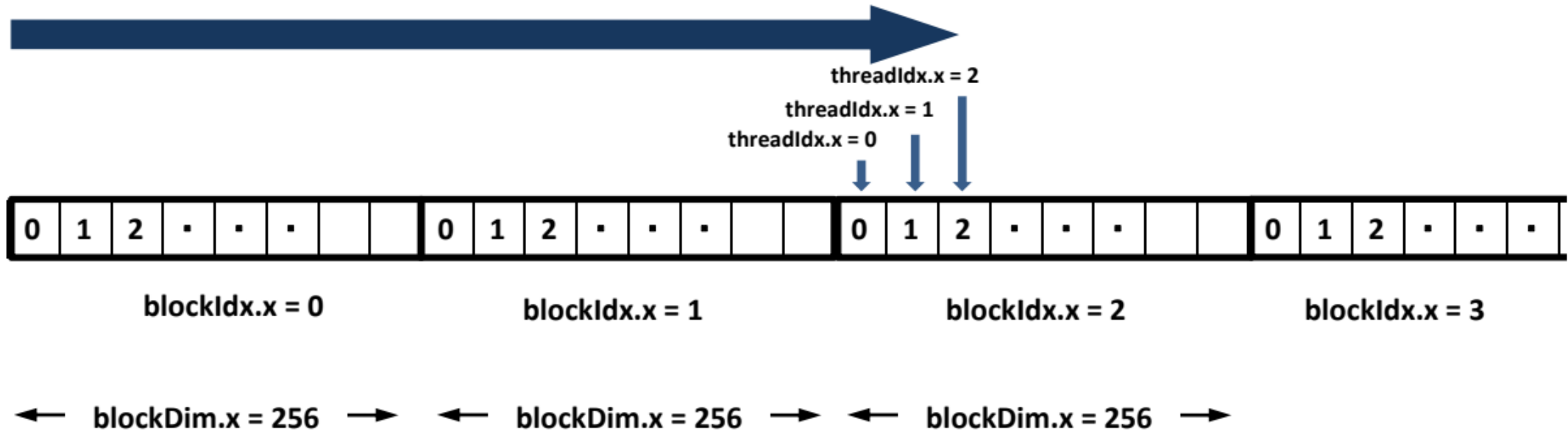
```
__global__ void g2g_copy
// =====
(
    double  *A,    // array pointer of the global memory
    double  *B      // array pointer of the global memory
)
// -----
{
    int    i = blockDim.x*blockIdx.x + threadIdx.x;

    B[i] = A[i];
}
```

병렬 데이터 액세스

- 1차원 배열 데이터에 대한 thread에서의 접근
- N개의 thread를 생성하고 1개의 thread가 배열의 한 요소에 접근하는 방식

$$i = \text{blockDim.x} * \text{blockIdx.x} + \text{threadIdx.x}$$



- Device code 안에서 선언하지 않고 인용만 가능, **재수정 불가**

gridDim	gridDim.x, gridDim.y, gridDim.z grid의 각 방향의 크기
blockIdx	blockIdx.x, blockIdx.y, blockIdx.z block의 각 방향의 index
blockDim	blockDim.x, blockDim.y, blockDim.z block의 각 방향의 크기
threadIdx	threadIdx.x, threadIdx.y, threadIdx.z thread의 각 방향의 index

▪ 함수형의 Qualifier(예약어)

__ global__ : device에서만 실행된다.
: host 측에서만 호출된다.

__ device__ : device에서만 실행된다.
: device에서만 호출된다.

__ host__ : host에서만 실행된다.
: host 측에서만 호출된다.
:(보통, CPU상에서의 프로그램의 함수로 특별히 선언할 필요가 없다).

C언어의 확장

● 계산 결과 회수

```
cudaMemcpy(b_h, h_d, n*sizeof(double), cudaMemcpyDeviceToHost);
```

Device에서 host 측에 데이터 전송

```
double sum = 0.0;
for(i= 0; i < n; i++) {
    sum += b_h[i];
```

계산 결과 확인

```
printf("Value= %8.6f\n",sum/(double)n);
```

● CPU에서의 실행

```
g2g_copy<<< grid, block>>>(a_d, b_d);
```



```
for(int j = 0; j < N; j++)
    b_h[j] = a_h[j];
```

배열 데이터 합산 kernel 함수

dim3 grid(n/256), block(256); ← block size을 256으로 함
add<<< grid, block>>>(a_d, b_d, c_d); ← kernel 함수 호출 실행

```
__global__ void add
// =====
(
    double  *A,    // array pointer of the global memory
    double  *B,    // array pointer of the global memory
    double  *C     // array pointer of the global memory
)
// -----
{
    int  i = blockDim.x*blockIdx.x + threadIdx.x;

    C[i] = A[i] + B[i];
}
```

배열 데이터 합산 kernel 함수

예: 산수 연습 문제

A[i] B[i]

$$14 + 34 =$$

$$8 + 50 =$$

$$22 + 92 =$$

$$33 - 23 =$$

$$- 5 + 40 =$$

$$- 1 + 70 =$$

$$17 + 3 =$$

$$100 - 40 =$$

$$6 + 25 =$$

A[i] B[i]

$$10 + 23 =$$

$$12 + 55 =$$

$$42 - 23 =$$

$$13 - 7 =$$

$$31 + 12 =$$

$$15 + 77 =$$

$$3 + 12 =$$

$$- 7 + 80 =$$

$$- 23 - 2 =$$

A[i] B[i]

$$29 - 31 =$$

$$- 2 - 20 =$$

$$51 - 44 =$$

$$- 39 + 28 =$$

$$- 27 + 10 =$$

$$- 9 - 62 =$$

$$30 - 38 =$$

$$78 + 22 =$$

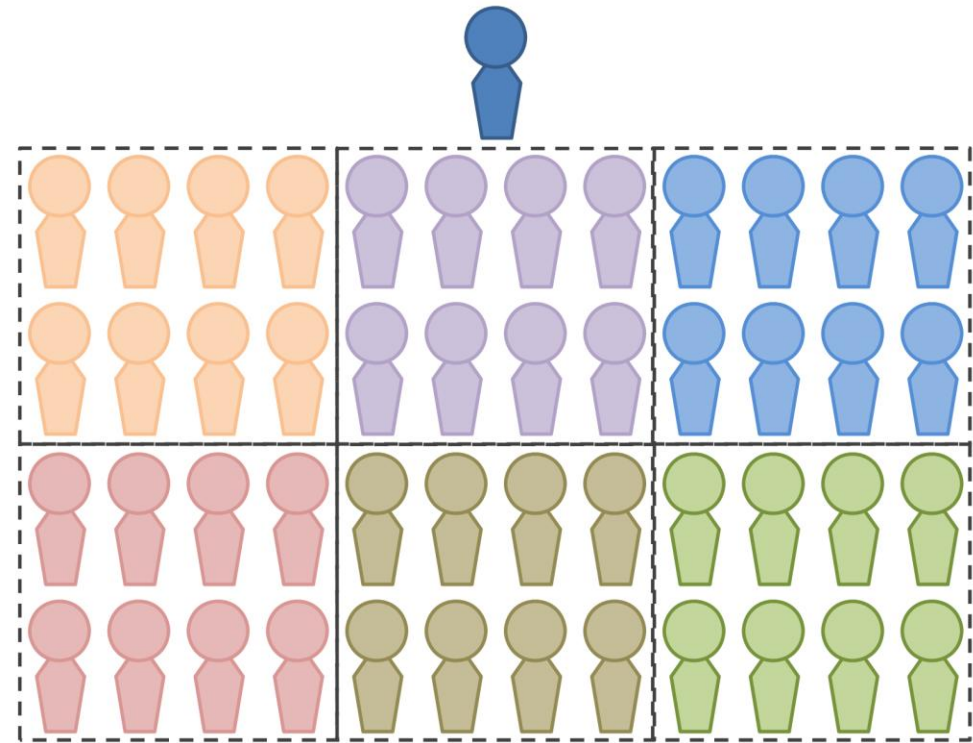
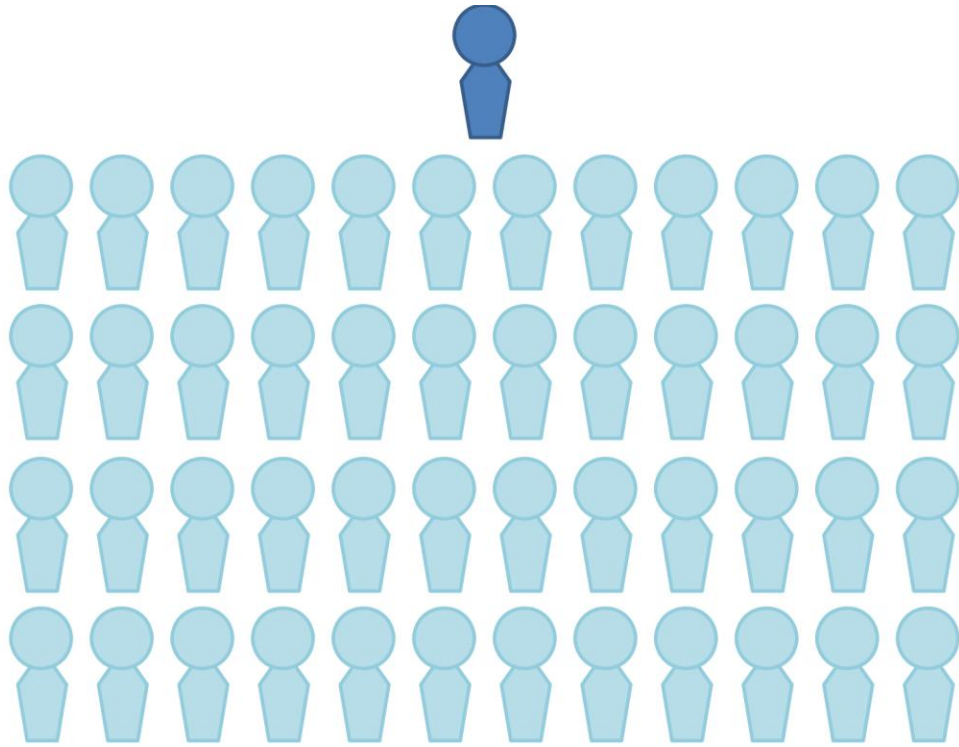
$$- 54 + 70 =$$

배열 데이터 합산 kernel 함수

산수 연습 문제 (1)

- 48명의 학생으로 구성된 학급에서 산수 훈련을 통해 계산 연습을 한다.
- 48개의 문제 중에서 학생들이 각각 다른 문제를 풀고 선생님에게 제출하기로 한다.
- 교사가 할 일:
 - 사전에 학생들에게 산수 드릴을 배부한다.
 - 학급 학생들을 학급으로 나눈다.
 - 문제 풀이 설명 프린트를 배포한다.
 - 채점 및 평균점수 산출.
- 학생이 해야 할 일:
 - 어떤 문제를 풀어야 하는지 지시를 듣는다.
 - 문제를 풀고 계산한다.
 - 몇 페이지의 문제를 풀었는지, 그 답을 답안지에 기입하고 이름을 써서 제출한다.
- 교사는 전교생에게 같은 지시만 할 수 있다고 한다.

클래스 반 편성(비유)



- **host code:** 선생님의 입장이 되어 많은 학생들이 문제를 풀도록 지시를 내린다. 조 편성을 결정한다.
- **device code:** 학생 개개인의 입장에서 무엇을 실행할 것인지를 기술한다.
- 클래스에 몇 반이 있는지, 자신이 몇 반에 속해 있는지, 반에 몇 명이 있는지, 자신이 몇 번째인지.

소요시간 측정 (1)

- 경과 시간을 측정함으로써 GPU Computing의 성능을 확인할 수 있고, 하드웨어 실행 모습을 상상할 수 있다.
- 또한, 튜닝을 위해 필수적이다.

계측범위



elapsedTime에 경과 시간
(msec)

```
cudaEvent_t start, stop;  
cudaEventCreate(&start);  
cudaEventCreate(&stop);  
float elapsedTime;  
  
cudaEventRecord(start,0);  
  
g2g_copy<<< grid, block>>>(a_d, b_d);  
  
cudaEventRecord(stop,0);  
cudaEventSynchronize(stop);  
cudaEventElapsedTime(&elapsedTime,start,stop);
```

소요시간 측정 (2)

- **CUDA Utility**를 이용한 시간 측정

- **CUDA_SDK_PATH**= **/usr/apps/free/NVIDIA_GPU_Computing_SDK/4.0/C**
- 컴파일 옵션 **-I \$CUDA_SDK_PATH /common/inc**
- 링크 옵션 **-L \$CUDA_SDK_PATH /lib -lcutil**

```
#include <cutil.h>
unsigned int timer;
```

```
cutCreateTimer(&timer);
```

```
cudaThreadSynchronize(); (이전 비동기 실행이 종료될 때까지 기다린다)
cutStartTimer(timer);      (gettimeofday() 를 사용하여 시간 측정 시작)
```



계측범위

```
cudaThreadSynchronize(); (이 기간 동안 비동기 실행이 종료될 때까지 기다림)
cutStopTimer(timer);      (gettimeofday() 를 이용한 시간 측정 종료)
```

```
double elapsed_time = cutGetTimerValue(timer); 경과 시간 (msec)
```

데이터 전송률 측정

● 오류 처리(API)

- `#define N (1024*1024*17)` 등으로 실행하면 결과가 엉망이 된다.
- CUDA의 API는 모두 return 값이 `cudaError_t` 타입의 `error status`를 반환하도록 되어 있다.

```
cudaError_t err = cudaMemcpy(...);  
if (err != cudaSuccess) {  
    fprintf(stderr, "Memcpy failed: %s.\n",  
        cudaGetErrorString(err));  
}
```

- 만약 `cudaMalloc`을 하지 않고 `cudaMemcpy()`를 실행한 경우 등은 `invalid device pointer`가 반환된다.

오류 처리 (kernel 함수)

- kernel 함수에는 return 값이 없다. `cudaGetLastError()`로 직전 에러를 수집하고, `cudaGetErrorString()`으로 메시지를 표시한다.

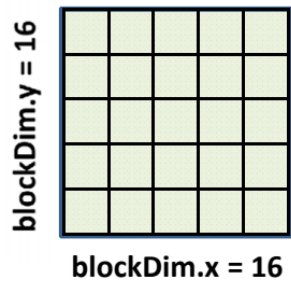
```
vec_add<<< , , , >>>(. . . );
cudaError_t err = cudaGetLastError();
if (err != cudaSuccess) {
    fprintf(stderr, "kernel launch failed: %s\n",
            cudaGetErrorString(err));
    exit(-1);
}
```

- `invalid configuration argument`가 표시된다. `grid.x = 65536`으로 최대값인 65535를 초과하고 있다.

2차원 데이터 액세스

- $NX \times NY$ 의 1차원 배열 데이터이지만, 2차원적으로 액세스

dim3 `grid(NX/16, NY/16),`
 `block(16, 16);`

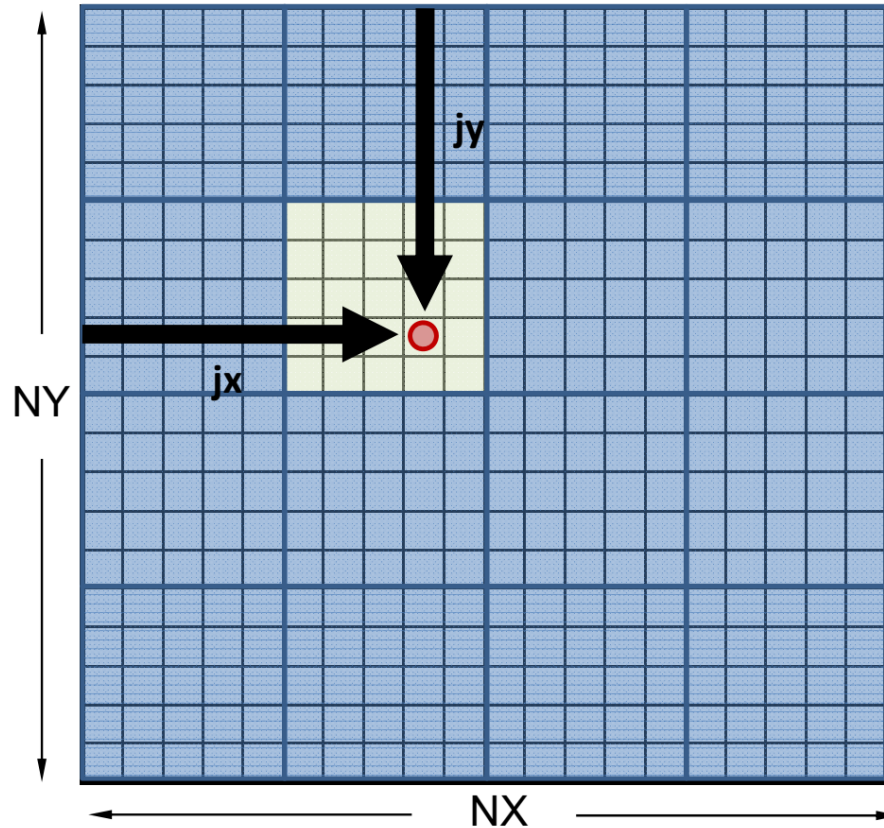


$jx = \dots$

$jy = \dots$

$j = \dots$

grid 크기 최대치
제한에서 벗어남



#define N (1024*1024*20)의 메모리 대역폭을 측정한다.

$C[i] = A[i] + B[i]$ 의 FLOPS 측정

- 시간 측정 방법을 이용하여
- `add<<<< grid, block>>>>(a_d, b_d, c_d)`의 소요 시간을 측정,
- 각 thread 의 연산 횟수가 1이므로 FLOPS를 계산한다.

```
__global__ void add
// =====
(
    double  *A,    // array pointer of the global memory
    double  *B,    // array pointer of the global memory
    double  *C     // array pointer of the global memory
)
// -----
{
    int  i = blockDim.x*blockIdx.x + threadIdx.x;

    C[i] = A[i] + B[i];
}
```


Device(디바이스 관리) API

- Device의 정보를 가져오는 API가 준비되어 있음.

`cudaGetDeviceCount(int *count)`

- CUDA가 동작하는 GPU의 개수를 반환한다.

`cudaSetDevice(int device_no)`

- 이후 실행을 device_no 의 GPU로 실행을 설정한다.

`cudaGetDevice(int *current_device)`

- 현재 지정된 GPU의 device 번호를 반환한다.

`cudaGetDeviceProperties(int *device,`

`cudaDeviceProp *prop)`

- deviceQuery와 같은 정보를 prop의 멤버로 가져옵니다.

- Tips: 이것들은 동일 노드에 여러 개의 GPU가 있는 경우 필수.

커널 함수 내 다차원 배열

- GPU 커널 함수 중 다차원 배열을 사용하고자 하는 경우가있다.

```
g2g_copy2d<<< Db, Dt >>>( );

__global__ void g2g_copy2d( )
{
    B[blockIdx.x][threadIdx.x][threadIdx.y]
      = A[blockIdx.x][threadIdx.x][threadIdx.y] ;
}
```

선형 메모리

cudaMallocPitch(), cudaMemcpy2D()를 사용해도, GPU 커널 함수에서 쉽게 다차원 배열을 사용할 수 없다. 프로그래밍으로는 권장하지 않지만, 다음과 같이 하면 가능하다.

```
__device__ double A[NY][NX], B[NY][NX];

__global__ void g2g_copy2d( )
{
    B[blockIdx.x][threadIdx.x][threadIdx.y]
      = A[blockIdx.x][threadIdx.x][threadIdx.y] ;
}
```

다차원 배열 데이터 전송

```
__device__ double a_d[NY][NX], b_d[NY][NX];  
static double a_h[NY][NX], b_h[NY][NX];
```

host에서 device로의 데이터 전송

```
cudaMemcpyToSymbol(a_d, a_h, sizeof(double)*NX*NY, 0);
```

device에서 host 로의 데이터 전송

```
cudaMemcpyFromSymbol(b_h, b_d, sizeof(double)*NX*NY, 0);
```

간접 참조를 통한 2차원 배열 확보

CPU의 경우, 동적 메모리 확보로 ****a_h**를 2차원 배열 **a_h[NY][NX]**의 크기로 사용할 수 있도록 한다.

```
double **a_h;  
  
a_h = (double **) malloc(sizeof(double *)*NY);  
  
for(j = 0; j < NY; j++) {  
    a_h[j] = (double *) malloc(sizeof(double)*NX);  
}
```

호스트에서의 2차원 배열에 대입

2차원 배열 `a_h[NY][NX]`의 요소에 난수를 대입한다.

```
for(j = 0; j < NY; j++) {  
    for(i = 0; i < NX; i++) {  
        a_h[j][i] = (double)rand() / RAND_MAX;  
    }  
}
```